Embedding Compression for Dense Retrieval: A Fair Empirical Comparison of Post-Hoc and Training-Time Methods

Sajil Awale

A THESIS

Submitted in partial fulfillment of the requirements
for the degree of Master of Science
in
The Department of Computer Science
to
The Graduate School
of
The University of Alabama in Huntsville

August 2026

Approved by:

Dr. Tathagata Mukherjee, Research Advisor/Committee Chair
Dr. Jacob Hauenstein, Committee Member
Dr. Manil Maskey, Committee Member
Dr. [3rd Committee Member Name], Committee Member
Dr. [Department Chair Name], Department Chair
Dr. Rainer Steinwandt, College Dean
Dr. Jon Hakkila, Graduate Dean

Abstract

Embedding Compression for Dense Retrieval: A Fair Empirical Comparison of Post-Hoc and Training-Time Methods

Sajil Awale

A thesis submitted in partial fulfillment of the requirements for the degree of Master of Science

Computer Science

The University of Alabama in Huntsville

August 2026

Modern retrieval-augmented generation (RAG) and large-scale search rely on dense embeddings, but storing billions of float32 vectors gets expensive fast. Three encoders, BGE-base-en-v1.5, RoBERTa-base, and MPNet-base, are fine-tuned on the same data and run through seven compression methods on NanoBEIR: INT8 quantization, binary quantization, Product Quantization (PQ), TurboQuant, Straight-Through Estimator (STE) binarization, Annealed Tanh binarization, and Matryoshka Representation Learning (MRL), along with stacked MRL+precision-reduction combinations. No single method comes out ahead across the board. PQ wins on BGE, which was pretrained for retrieval to begin with. RoBERTa and MPNet prefer training-time binarization, and annealed tanh stacks with MRL better than STE does. MRL+PQ and MRL+TurboQuant stay usable out to 96× compression, though method rankings agree only up to 8× and start to diverge at 32×. Everything is reported as Pareto curves so a practitioner with a fixed memory budget can read the choice straight off the plot.

Acknowledgements

I'm very thankful to my advisor, Dr. Tathagata Mukherjee, for all the help, advice, and encouragement he gave me while I was doing this study. It was his knowledge and experience that helped guide the direction of this work and helped me grow as a researcher.

I also want to thank Dr. Jacob Hauenstein and Dr. Manil Maskey, who were on my thesis committee, for their helpful comments and criticisms. Their thoughtful suggestions made this work much better.

I acknowledge NASA IMPACT for giving the computing resources that made the experiments in this thesis possible. I'm also indebted to Nishan Pantha and Muthukumaran Ramasubramanian, two members of the LLM team, for their support and helpful discussions during this study.

Lastly, I'm deeply grateful to my family and friends for always being there for me and listening to my crazy ideas. The fact that they believed in me kept me going through every struggle.

Table of Contents

List of Figures

List of Tables

Chapter 1. Introduction

1.1 Motivation

Dense vector retrieval has found many applications in modern AI systems, such as Retrieval Augmented Generation (RAG) [20], recommendation systems, question answering services, and more. All of these applications depend on its ability to find semantically relevant documents. Unlike sparse retrieval methods like TF-IDF or BM25, which are based on exact keyword matching, it works by encoding queries and corpus documents into continuous vector embeddings using bi-encoder architectures [26, 17]. Similarity between texts is computed using cosine similarity.

When scaling such systems, memory becomes one of the primary bottlenecks. Consider using a dense retrieval model like BGE-base-en-v1.5 [39], which produces a 768-dimensional float32 vector per passage. Each vector would take 3,072 bytes. For a billion passages, the embeddings alone would grow to approximately 2.9 TB.

Usually, dense retrieval systems, such as those built with libraries like FAISS [15], require all the embedding indices of corpus documents to live in Random Access Memory (RAM). This in-memory requirement is essential for low-latency systems. For random read access, an NVMe SSD takes about 100

microseconds, which is a thousand times slower than DRAM. To serve such a dense retrieval system in the cloud, it is practical to choose memory-optimized cloud compute instances like the AWS x2gd family. An AWS x2gd instance costs about $3.80 per GB per month [2]. Table 1.1 shows how this cost would scale with different embedding dimensions and corpus sizes for the commonly used models.

Table 1.1: Float32 embedding index size and estimated monthly cost on memory-optimized AWS x2gd instances ($3.80/GB/month) at two corpus scales [29, 2]. Sizes are reported in GB (SI).

Dim	Example Models	100M Passages	1B Passages
384	all-MiniLM-L6-v2, bge-small-en-v1.5	143 GB / $543/mo	1,431 GB / $5,435/mo
768	bge-base-en-v1.5, nomic-embed-text-v1	286 GB / $1,087/mo	2,861 GB / $10,871/mo
1024	bge-large-en-v1.5, mxbai-embed-large-v1	381 GB / $1,449/mo	3,815 GB / $14,495/mo
1536	text-embedding-3-small	572 GB / $2,174/mo	5,722 GB / $21,743/mo
3072	text-embedding-3-large	1,144 GB / $4,348/mo	11,444 GB / $43,487/mo

For our previous example, a billion passages using an encoder producing a 768-dimensional vector would cost $10,800 monthly. But better models tend to produce higher-dimensional vectors, which worsens the problem. For instance, State of the Art (SOTA) encoder models from OpenAI, like text-embedding-3-large, produce 3072-dimensional vectors, which would cost $43,487 per month.

Apart from the cost factor, larger embedding sizes invite more problems. Every query vector should be compared against each passage vector from memory. So, a bigger vector means our retrieval system can serve fewer queries per second.

Furthermore, for resource-constrained environments such as on-device or edge deployment, having multi-terabytes of memory wouldn't be feasible regardless of the budget. Therefore, embedding compression is a necessity for deploying dense retrieval at scale.

Even simple compression techniques already show real promise. Transforming the embedding with binary quantization (mapping negative values to 0 and positive values to 1) replaces each float32 value with a single bit. Surprisingly, this aggressive compression retains most of the original retrieval quality [29]. There are many studies on embedding compression, which are discussed in Chapter 2. Compression can be stacked to achieve even better compression ratios. But how far can these embeddings be compressed before the retrieval quality degrades substantially? And does incorporating fine-tuning (training-time compression) of the models result in better representations than compression applied to a pre-trained model without retraining (post-hoc compression)? This thesis investigates these questions.

1.2 Problem Statement

There is a fundamental conflict between compression ratio and retrieval quality. Information is exchanged for a lower memory footprint in every compression technique, and the nature of this trade-off varies depending on the method and compression level.

This trade-off has started to be systematically mapped in recent work. [6] systematically evaluate retrieval performance at different compression ratios using

a variety of post-hoc compression techniques. Nevertheless, this comparison has not yet been extended to training-time compression. In most prior work [40, 11, 29, 1], each approach is evaluated using different base models, training corpus, and benchmark. Therefore, a strategy that seems to succeed may be due to a better starting point rather than to a better compression strategy.

Furthermore, stacking is an additional gap. Combinations of several complementary compression methods, such as dimensionality reduction with quantization, remain largely unexplored. The previous research that identifies both gaps is reviewed in Chapter 2.

This thesis uses controlled empirical experimentation to fill in both gaps. Results are reported as Pareto frontiers on the NanoBEIR benchmark [34]. A Pareto frontier is the set of configurations where no alternative achieves both higher retrieval quality and higher compression. This directly answers the question: which strategy should you use given a memory budget? Section 1.4 goes into detail on the contributions.

1.3 Research Questions

This thesis is organized around four research questions:

1. **Post-hoc versus training-time compression.** Does training-time compression lead to better retrieval performance than post-hoc compression at equivalent memory budget?

2. **Straight Through Estimator (STE) vs annealed tanh for binary embedding training.** To backpropagate through the non-differentiable sign function when training binary embeddings, you need a surrogate gradient. Does the choice between the Straight Through Estimator (STE) and annealed tanh affect retrieval quality?

3. **Stacking complementary compression techniques.** Does using multiple complementary compression methods produce better retrieval performance than using just a single technique individually, in terms of compression ratio trade-offs?

4. **Cross-model generalizability.** Given that the encoder models have different starting retrieval quality, are the findings of compression the same, or are they different from one model to another?

1.4 Contributions

This thesis makes the following contributions to the field of efficient dense text retrieval:

1. **Systematic cross-method comparison.** Post-hoc and training-time compression are evaluated under identical training data and benchmarks. Both single and stacked combinations were evaluated at various compression ratios. Previous work evaluated their methods in distinct setups that are difficult to compare against each other. This study provides a direct and fair comparison.

2. **Binarization training strategy comparison.** We compare two gradient estimate methods, STE and annealed tanh, for the first time directly under the same conditions [4, 40].

3. **Stacking complementary compression techniques.** Matryoshka Representation Learning (MRL), a dimensionality reduction, is combined with five quantization methods and evaluated systematically. These techniques include both post-hoc and training-time methods.

4. **Reproducible experimental pipeline.** The training code base for training-time compression was made public. It supports multi-GPU training across multiple retrieval datasets with source-weighted batch sampling. We also released the evaluation codebase, which caches the embeddings and performs any compression transform at evaluation time. All results are viewable through an interactive dashboard.

1.5 Thesis Organization

The rest of this paper is organized as follows. Chapter 2 gives the background and related works. It covers bi-encoder dense retrieval, contrastive learning objectives, the embedding compression methods studied in this thesis, and Information Retrieval (IR) evaluation metrics. Then it reviews what has already been done and finds the gaps that lead to the research questions. Chapter 3 describes the datasets. It shows the construction of the multi-source training corpus and NanoBEIR benchmark.

Chapter 4 sets out the methodology. It presents the experimental design, the shared training setup and float32 baseline, the training-time and post-hoc compression methods, the stacking combinations of compression, and the framework for evaluating the results.

Chapter 5 reports the findings and their analysis. It compares post-hoc, training-time, and stacked methods across three encoder backbones. It also presents Pareto frontier analysis. Finally, in Chapter 6, it reviews the results against the initial set of research questions and provides directions for future work.

Chapter 2. Background and Related Work

This chapter covers the technical background for this thesis. Section 2.1 describes the bi-encoder architecture for dense retrieval and the three models we used. Section 2.2 introduces the contrastive learning objective for training the embedding model. Section 2.3 covers the embedding compression techniques. In Section 2.4, we define the evaluation metrics and the significance test. And finally, Section 2.5 looks at the related work and positions this thesis relative to previous contributions.

2.1 Dense Retrieval with Bi-Encoders

2.1.1 Transformer Encoders

Modern dense retrieval systems are built on top of transformer-based language models [9]. A transformer encoder maps a token sequence of length L to a sequence of contextualized hidden states $\mathbf{H} \in \mathbb{R}^{L \times d_h}$, where d_h is the hidden dimension, through a stack of self-attention and feed-forward layers. Each self-attention layer allows every token to attend to every other token in the sequence, producing representations that capture long-range syntactic and semantic dependencies. Bidirectional encoders such as BERT (Bidirectional Encoder Representations from Transformers) [9] are pre-trained on large text corpora via masked

language modelling (MLM), yielding general-purpose representations that transfer well to downstream tasks with minimal task-specific fine-tuning.

2.1.2 Bi-Encoder Architecture

A bi-encoder, sometimes called a dual encoder, is made up of two transformer encoders: one for passages and one for queries. The outputs of these two encoders are compressed into dense vectors of fixed size [26, 17]. The full bi-encoder retrieval process is shown in Figure 2.1. Most of the time, the two towers share weights. There is a separate encoder for each question (q) and passage (p). Each query q and passage p are encoded independently:

$$\mathbf{e}_q = \text{pool}(\text{Enc}(q)), \qquad \mathbf{e}_p = \text{pool}(\text{Enc}(p)), \tag{2.1}$$

where pool($\cdot$) is a pooling operation applied to the set of hidden states. Sentence-BERT (SBERT) [26] established mean pooling as a standard for retrieval models. Relevance is computed using cosine similarity between the query and passage embeddings:

$$\text{sim}(\mathbf{e}_q, \mathbf{e}_p) = \frac{\mathbf{e}_q \cdot \mathbf{e}_p}{\|\mathbf{e}_q\|\|\mathbf{e}_p\|}. \tag{2.2}$$

Separating the query and passage encoding is a key feature of bi-encoders for large-scale retrieval. Passage embeddings can be calculated offline and stored in an approximate nearest-neighbor (ANN) index. This way, only the query encoder runs at query time, and retrieval is reduced to a single nearest-neighbor

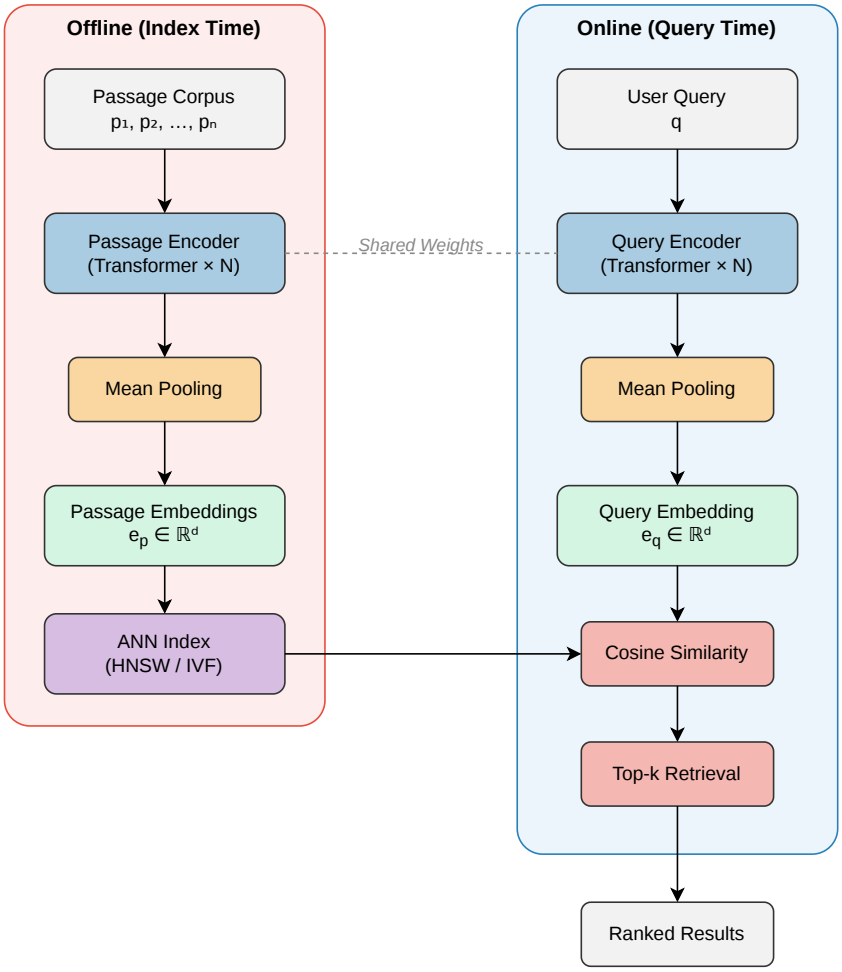

Figure 2.1: Bi-encoder retrieval architecture. Everything in the corpus is encoded and saved in an ANN index at index time (left). At query time (on the right), the query is encoded and compared to the index using cosine similarity to find the k most relevant passages. Weights are shared by both encoders.

lookup [17]. Cross-encoders, on the other hand, jointly encode both the query and passage and hence cannot precompute a representation.

For an exact exhaustive search, each query is compared against all passage vectors. This takes $\mathcal{O}(N \cdot d)$ time. Here, N is the number of passages and d is the embedding dimension.

Take, for example, 10 million passages each with 768-dimensional embeddings. Computing relevant passages for a query takes about 15 billion floating-point operations. This makes a latency of less than 1 millisecond unachievable. Hierarchical Navigable Small World (HNSW) [23] and Inverted File Index (IVF) [15] are two ANN indices that address this problem. They do this by organizing vectors so that only a small part of the corpus needs visiting. This gets the speed back with a small loss in recall. Specialized disk-resident indices like DiskANN [33] lower this memory need even more by using SSDs to improve graph traversal. But their latency is roughly three orders of magnitude slower than that of DRAM. Hence, in-memory indexing is preferred for interactive workloads. Even with ANN indexing, most applications need the full set of passage embeddings in memory. This thesis gets around the memory problem by compressing the embeddings.

2.1.3 Backbone Models

This thesis evaluates compression methods across three backbone encoders to test whether the compression technique is effective and if it generalizes for different backbones or not.

BGE-base-en-v1.5 [39] is the primary backbone for this thesis. BGE (BAAI General Embedding) is a family of English bi-encoders from the Beijing

Academy of Artificial Intelligence (BAAI). The base variant was chosen because it had strong performance on the Massive Text Embedding Benchmark (MTEB) [24] at moderate parameter count.

Table 2.1: Backbone encoder models. All three of them use a 12-layer transformer block and produce 768-dimensional float32 embeddings by mean pooling.

Model	**Parameters**	**Pre-training Strategy**
BGE-base-en-v1.5 [39]	109M	MLM + retrieval fine-tuning
RoBERTa-base [21]	125M	Optimized MLM
MPNet-base [31]	110M	Masked + permuted LM

RoBERTa-base (Robustly Optimized BERT Pretraining Approach) [21] is a replication study of BERT that corrects many under-training choices. It gets rid of the Next Sentence Prediction (NSP) objective, uses dynamic masks, bigger batch sizes, and 10 times more training data. Even though the architecture stays the same, these changes make the representations better on standard NLU benchmarks [37, 25].

MPNet-base (Masked and Permuted Pre-training) [31] combines masked language modelling with permuted language modelling. When BERT predicts a masked token, it ignores other masked places. MPNet, on the other hand, predicts tokens autoregressively in a random permutation order while paying attention to the whole sequence. This way, it finds dependencies between masked tokens that MLM-only models miss. The MPNet pre-training objective is different from both

BERT and BGE. This helps test the generalizability of the compression results even more.

The three backbones have different starting retrieval quality. BGE is the best place to start because it comes already fine-tuned for retrieval. MPNet has an intermediate starting point because of its masked and permuted goal. RoBERTa has the weakest initial retrieval representations because it was only taught on the MLM task. This order (BGE > MPNet > RoBERTa) is also confirmed by the zero-shot NanoBEIR baselines reported in Chapter 4. With these different qualities of backbones, we are trying to answer whether the property of compression trade-offs stays the same or not.

2.2 Contrastive Learning for Dense Retrieval

Contrastive learning teaches a model to put things that are semantically related close to each other, and things that are not, far apart [35]. In the Information Noise-Contrastive Estimation (InfoNCE) loss, this is turned into an N-way classification task: the model has to give the positive pair the highest similarity score among the $N-1$ negatives.

Multiple Negatives Ranking Loss (MNRL) is the in-batch implementation of InfoNCE used by the sentence-transformers system [26]. It is used in this thesis. The negatives are obtained for free from other examples within the batch. For a batch of N positive query-passage pairs $\{(q_i, p_i^+)\}_{i=1}^N$, the positive passage of other pairs acts as a negative for query q_i (Figure 2.2).

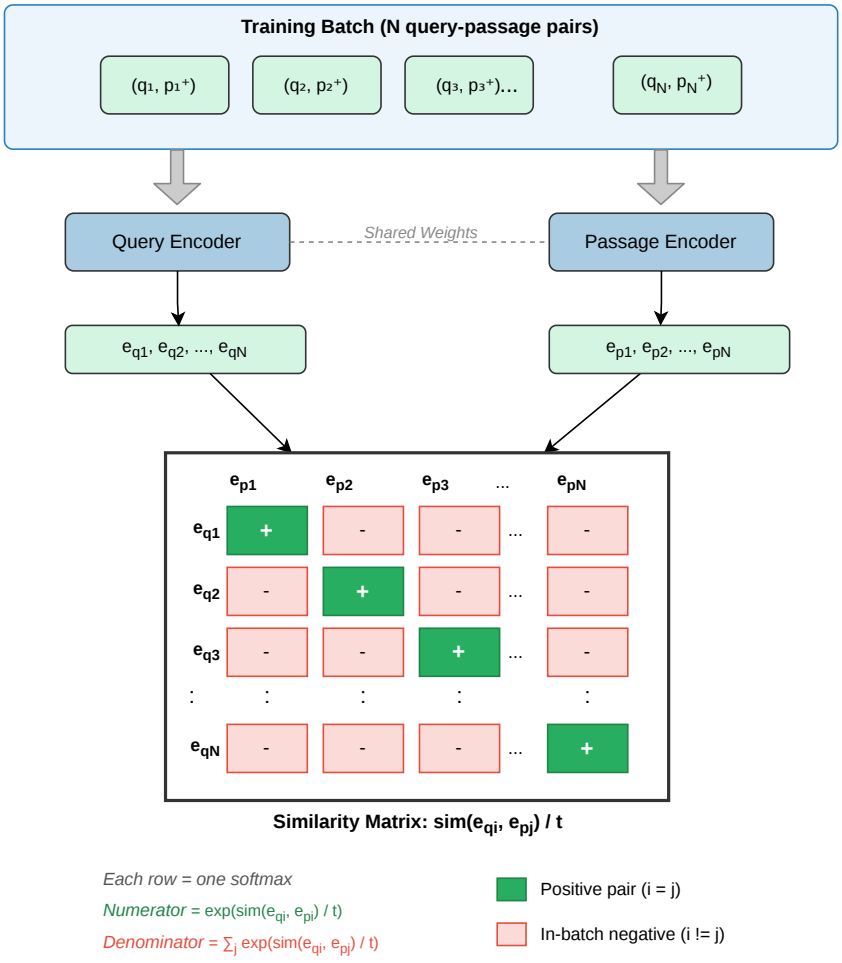

Figure 2.2: Multiple Negatives Ranking Loss. A training batch of N pairs is encoded by the shared weights. The $N \times N$ similarity matrix scores every query against every passage in the batch. The loss is made up of positive pairs on the diagonal (green), in the numerator. The denominator is made up of each full row, which makes each row an N-way softmax classification.

The loss for the full batch is:

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp\Big(\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{p_i^+})/\tau\Big)}{\sum_{j=1}^{N} \exp\Big(\text{sim}(\mathbf{e}_{q_i}, \mathbf{e}_{p_j^+})/\tau\Big)}, \tag{2.3}$$

where $\tau > 0$ is a temperature hyperparameter that controls how peaked the softmax distribution is. In-batch negatives are effective because large batch sizes expose each query to many diverse negatives simultaneously. Also, batch mates are often related by topic to the query, providing challenging but informative negatives without any explicit negative mining [17].

2.3 Embedding Compression Methods

This thesis organizes embedding compression methods along two orthogonal axes. The first axis is when compression is applied. Post-hoc compression methods are applied to the embeddings generated by the models that weren't modified. But training-time methods incorporate compression awareness into the model's fine-tuning objective. The second axis is how the memory requirement is reduced. We can reduce precision of embeddings by decreasing the number of bits per dimension, we can truncate embedding dimensions, or we can learn a hash that fits a codebook mapping embeddings to discrete codes. Table 2.2 presents the methods studied in this thesis along these two axes.

The classification in Table 2.2 reflects when training is modified, not when the compressed representation is produced. At retrieval time, all of the methods do something: truncation, sign-thresholding, or scalar quantization. For post-hoc methods, this retrieval time operation is the entire compression step, applied to a model that was not trained for it. For training-time methods, the same kind of operation is still required at retrieval (taking the sign of a BAT embedding, truncating an MRL embedding to its first m dimensions). But they are effective

Table 2.2: Organization of embedding compression methods based on when and how compression is applied. Abbreviations: INT8 = 8-bit integer quantization; PQ = Product Quantization; BAT = Binary-Aware Training (STE and Annealed Tanh are two training variants); STE = Straight-Through Estimator; MRL = Matryoshka Representation Learning.

	Precision Reduction	**Dim. Reduction**	**Learning to Hash**
Post-hoc	INT8, Binary, TurboQuant	—	PQ
Training-time	BAT (STE / Annealed Tanh)	MRL	—

because training was modified to make that operation lossy in a controlled way rather than destructive.

The following subsections describe each method, grouped by when compression is applied.

2.3.1 Post-hoc Methods

2.3.1.1 Scalar Quantization

As shown in Figure 2.3, scalar quantization maps each float32 value to a lower-precision number. To find the lower and upper limits $[\min_j, \max_j]$ for INT8 quantization, we need a calibration subset. Here, j is the index for the embedding dimension. Embedding value x at index j is then quantized as:

$$x_q = \left\lfloor \frac{x - \min_j}{\delta_j} \right\rceil - 128, \qquad \delta_j = \frac{\max_j - \min_j}{255}, \tag{2.4}$$

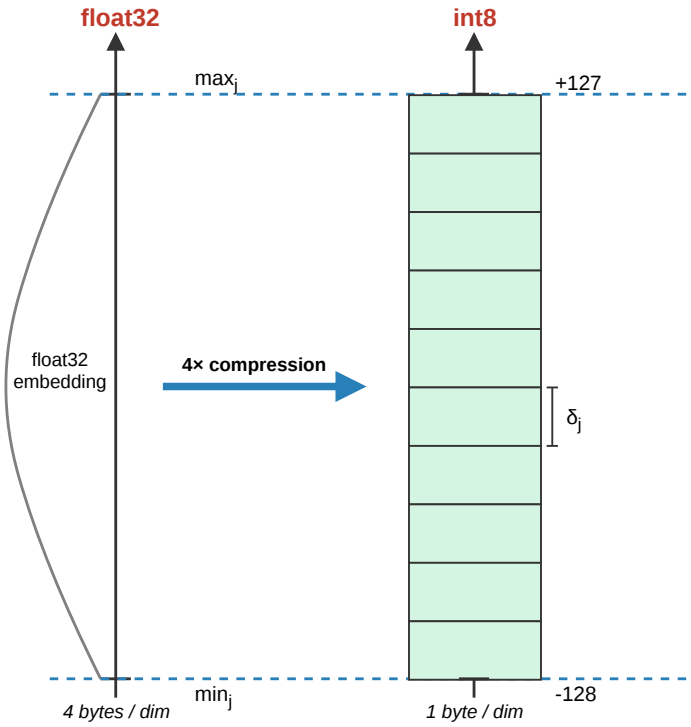

Figure 2.3: Scalar quantization of a single embedding dimension. The float32 value x (left) is mapped to a signed 8-bit integer $x_q \in [-128, 127]$ (right) using 256 uniform quantization levels of width δ_j, and per-dimension calibration range $[\min_j, \max_j]$.

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer, and the -128 shift turns the unsigned range $[0, 255]$ into the signed 8-bit range $[-128, 127]$. It cuts storage from 4 bytes per dimension to 1 byte per dimension ($4\times$ compression) by using all 256 levels across the recorded range of each dimension [10].

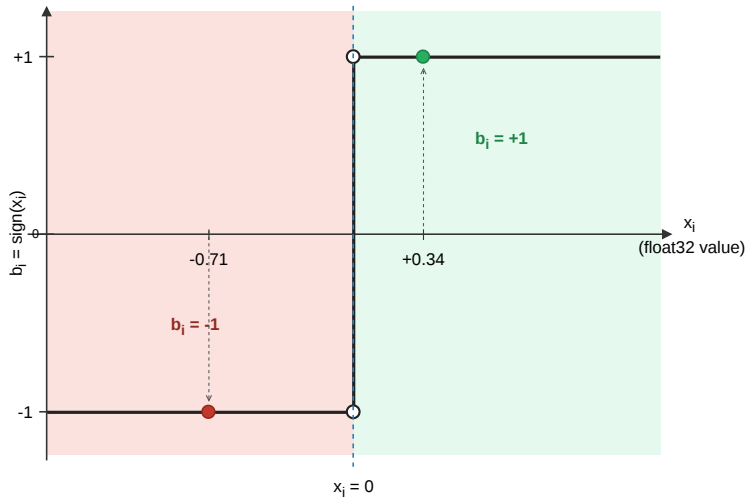

Figure 2.4: Binary quantization maps each float32 embedding dimension x_i to a binary value $b_i \in \{-1, +1\}$ with the sign function. All positive values map to $+1$ (green region) and all negative values map to -1 (red region).

2.3.1.2 Binary Quantization

By using the sign function, binary quantization changes each float32 value to a single bit, as shown in Figure 2.4:

$$b_i = \text{sign}(x_i) \in \{-1, +1\}, \tag{2.5}$$

where x_i is the i-th dimension of the float32 embedding and b_i is the corresponding bit. This gives a 32× compression over float32. The Hamming distance is used to find the similarity between a binary query vector $\mathbf{b}_q$ and a passage vector $\mathbf{b}_p$. In modern hardware, the Hamming distance can be reduced to bitwise XOR followed by popcount, which can be much faster than cosine similarity computation [11].

2.3.1.3 Product Quantization

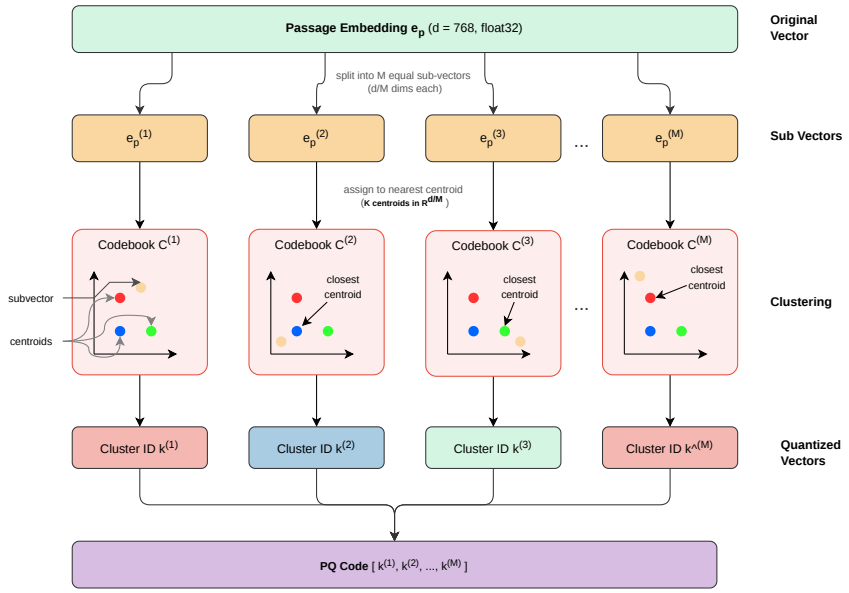

Figure 2.5: Product quantization encoding pipeline. Each passage embedding $\mathbf{e}_p \in \mathbb{R}^d$ is split into M equally sized sub-vectors. The sub-vector is assigned to the closest centroid in its sub-space codebook. The codebook is fitted offline using k-means on the corpus. The compact PQ code is made up of the M centroid indices. It cuts storage space from $d \times 4$ bytes to $M \times \lceil \log_2 K \rceil$ bits.

High compression is possible with product quantization (PQ) [14]. It does this by splitting each d-dimensional embedding into M equal sub-vectors and giving each sub-vector a single integer number instead of raw floats. The full encoding process is shown in Figure 2.5. A set of K centroids is fit to the text embeddings by k-means for each of the M subspaces. Then, the value of the closest centroid is put in place for each corpus sub-vector $\mathbf{e}^{(m)} \in \mathbb{R}^{d/M}$:

$$\hat{k}^{(m)} = \arg\min_k \left\| \mathbf{e}^{(m)} - \mathbf{c}_k^{(m)} \right\|. \tag{2.6}$$

A passage is saved as a list of M numbers $(\hat{k}^{(1)}, \ldots, \hat{k}^{(M)})$, and each one needs $\log_2 K$ bits. A float32 embedding with 768 dimensions goes from 3,072 bytes to 64 bytes when $M = 64$ sub-vectors and $K = 256$ centroids. This is a compression ratio of 48×.

When the data is retrieved, the learned centroids are used to decode the corpus embeddings from their PQ codes back to their approximate float32 representations. Query embeddings, on the other hand, are kept at full float32 precision. After that, scoring is done by finding the standard cosine similarity between the rebuilt corpus and the query vectors. Asymmetric reconstruction, which only compresses the passage side and keeps the query accurate, brings back a lot of the quality that was lost during compression [15].

2.3.1.4 TurboQuant

TurboQuant [43] is a post-hoc method for reducing precision that works through three stages to get near-optimal rate-distortion performance. In the first stage, each embedding is rotated randomly (random Hadamard transform, $O(d \log d)$). The rotation keeps all pairwise distances and inner products the same, but the distribution of each coordinate transforms to a beta distribution, which means we can use a single quantizer for all dimensions. Second, a Lloyd-Max quantizer sets non-uniform bucket boundaries that reduce the mean squared error (MSE), bringing distortion within 2.7 times the Shannon lower bound. Third, because Lloyd-Max reconstruction introduces bias in inner product estimation, TurboQuant only allocates $b - 1$ bits for MSE-optimal quantization and saves 1

bit for Quantized Johnson-Lindenstrauss (QJL). QJL fixes this bias. At retrieval time, the stored codes are decoded into nearby float32 vectors, and standard cosine similarity is used.

2.3.2 Training-Time Methods

2.3.2.1 Straight-Through Estimator Binarization

By adding the sign function to the model during fine-tuning, post-hoc binary quantization could be made a lot better. The main problem is that the sign function is not differentiable at zero and has zero derivative everywhere else. This means that normal backpropagation can't happen. To fix this, the Straight-Through Estimator (STE) [4] uses the sign function in the forward pass and keeps the gradient the same in the backward pass, as if the operation were the identity function:

$$\text{forward:} \quad \hat{\mathbf{e}} = \text{sign}(\mathbf{e}), \qquad \text{backward:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{e}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{e}}}, \tag{2.7}$$

where $\mathbf{e}$ is the embedding before binarization, and $\hat{\mathbf{e}}$ is its binarized form. There is a bias in the estimator because the passed gradient is not the true gradient of the sign function. However, Bengio et al. report that it works better than more mathematically sound alternatives. When the STE is used to train the model, it encourages representations close to ± 1. This way, the quantization step loses as little information as possible at the time of inference.

2.3.2.2 Annealed Tanh Binarization

The STE approximates the gradient of the sign function poorly. Annealed Tanh binarization replaces the STE with a smooth surrogate that gets sharper over time until it matches the sign function [5]:

$$\hat{\mathbf{e}} = \tanh(\beta \cdot \mathbf{e}), \tag{2.8}$$

where β is scheduled as $\beta_t = \sqrt{\gamma \cdot t + 1}$, t is the global optimizer step and γ is a growth rate hyperparameter [40]. When training begins ($t = 0$), $\beta = 1$ and $\tanh(\mathbf{e}) \approx \mathbf{e}$ for small activations. This puts the model in a float32 domain with few restrictions. As training goes on, β increases and $\tanh(\beta \cdot \mathbf{e})$ gets closer to $\text{sign}(\mathbf{e})$, making the binarization constraint tighter over time. The hard sign function is used at inference time to get back to a normal binary representation. The annealing schedule gives the model time to change how it represents things before the hard constraint is enforced. This progress is shown in Figure 2.6 over six training stages.

2.3.2.3 Matryoshka Representation Learning

Matryoshka Representation Learning (MRL) [18] trains a model to concentrate the most important semantic information within the leading dimensions of the embedding vector. The main idea is that if you train the model to solve the retrieval task at different embedding truncation at the same time, the first

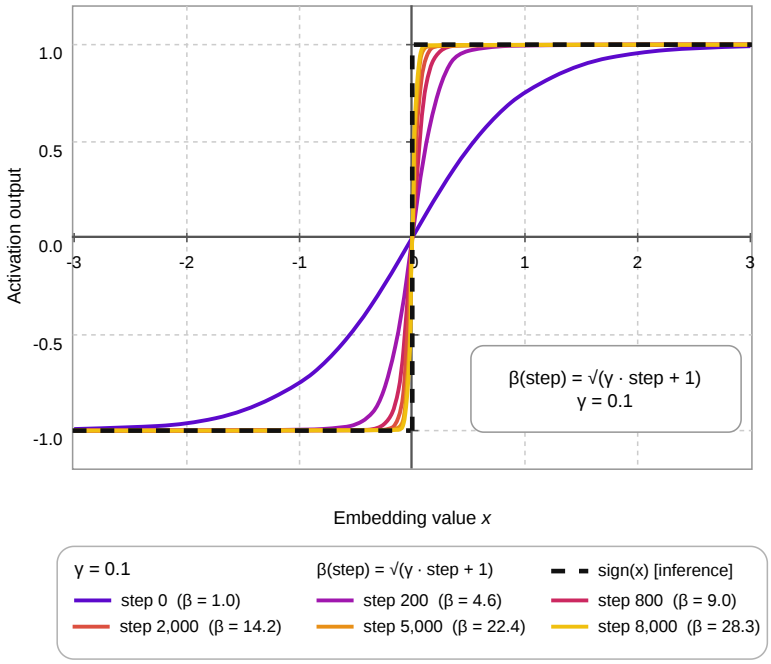

Figure 2.6: Annealed Tanh binarization surrogate $\tanh(\beta_t \cdot x)$ at six training checkpoints with $\beta_t = \sqrt{\gamma \cdot t + 1}$ and $\gamma = 0.1$. At step 0 ($\beta = 1.0$), the surrogate is almost straight. At step 8,000 ($\beta = 28.3$, dashed), it becomes more like the hard sign function, which is used at inference time.

few levels will pick up the most critical information, while the higher levels add more and more fine-grained details.

As shown in Figure 2.7, MRL adds to the standard MNRL goal by computing losses at a fixed set of nested dimensions $\mathcal{M} = \{m_1, m_2, \ldots, m_K\}$, where $m_1 < m_2 < \cdots < m_K = d$. This is the multi-granularity loss:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}_{\text{MNRL}}(\mathbf{e}_{1:m}), \tag{2.9}$$

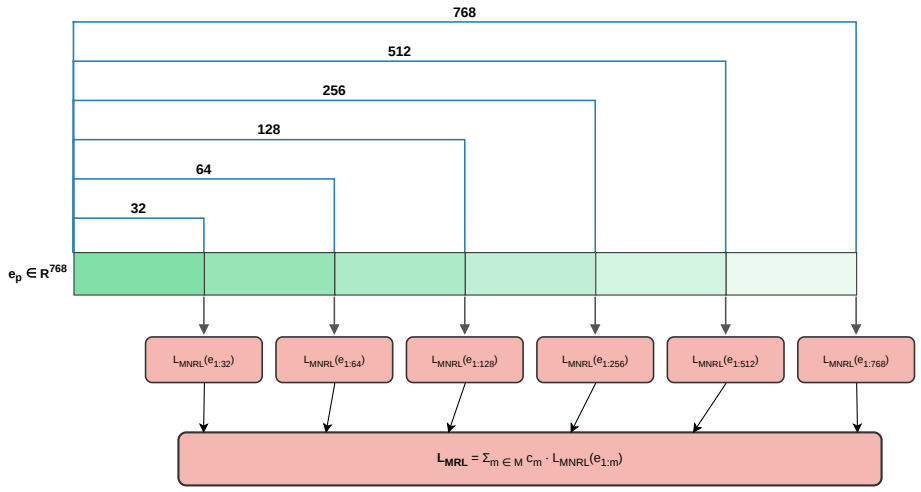

Figure 2.7: MRL training. Each prefix $\mathbf{e}_{1:m}$ for $m \in \mathcal{M} = \{32, 64, 128, 256, 512, 768\}$ is cut off from the passage embedding $\mathbf{e}_p \in \mathbb{R}^{768}$. There is a different MNRL loss for each truncation, and the total loss, $\mathcal{L}_{\text{MRL}}$, is equal to the sum of those losses (Equation 2.9).

where c_m are positive scalar weights. The MNRL loss $\mathcal{L}_{\text{MNRL}}(\mathbf{e}_{1:m})$ is found by using only the first m dimensions. We use $\mathcal{M} = \{32, 64, 128, 256, 512, 768\}$ for all MRL studies in this thesis. The full embedding dimension is 768, which is equal to d.

At the time of inference, MRL lets the embedding be cut down to any number of dimensions ($m \in \mathcal{M} \setminus \{d\}$) without needing any extra fine-tuning. This makes it possible to directly reduce the number of dimensions at runtime. This is what makes MRL embeddings truly different from standard embeddings. Truncating a standard embedding will destroy the quality of the retrieval, but an MRL-trained embedding is made to stay useful at every supported truncation.

2.4 Information Retrieval Evaluation

2.4.1 Ranking Metrics

Retrieval systems are evaluated by how well they rank relevant documents above irrelevant ones. All the metrics used in this thesis are based on the idea that each corpus document is either relevant to a given query or not. The retrieval system makes a ranked list of documents for each query. Metrics are calculated at cut-off k, which means that only the top k documents in the ranked list are considered.

Mean Reciprocal Rank (MRR@k) [36] is the primary metric used in this thesis. It measures the position of the first relevant document in the ranked list:

$$\text{MRR@}k = \frac{1}{\text{rank}_1}, \quad \text{where } \text{rank}_1 = \min\{i \leq k : \text{doc}_i \in \mathcal{R}\}, \tag{2.10}$$

and $\text{MRR@}k = 0$ if the top k doesn't have any relevant documents. MRR is very sensitive to the position of the single best hit, and it works well for retrieval tasks where there is only one right answer passage per query, which is the case for most NanoBEIR subsets.

Normalized Discounted Cumulative Gain (NDCG@k) [13] measures ranking quality by assigning higher credit to relevant documents appearing

earlier in the list. The DCG at position k is:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\mathbf{1}[\text{doc}_i \in \mathcal{R}]}{\log_2(i+1)}, \tag{2.11}$$

where $\mathcal{R}$ is the set of relevant documents and $\mathbf{1}[\cdot]$ is the indicator function. NDCG@k normalizes DCG@k by the ideal DCG (IDCG@k), which is the score that you get when all relevant documents are ranked first:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \qquad \text{IDCG@}k = \sum_{i=1}^{\min(|\mathcal{R}|,\, k)} \frac{1}{\log_2(i+1)}. \tag{2.12}$$

NDCG@k lies between 0 and 1, with 1 being the best possible score.

Recall@k is the fraction of the number of relevant documents that show up in the top k results:

$$\text{Recall@}k = \frac{|\{i \le k : \text{doc}_i \in \mathcal{R}\}|}{|\mathcal{R}|}. \tag{2.13}$$

Recall@k is limited by $\min(k, |\mathcal{R}|)/|\mathcal{R}|$ and only equals 1 when all relevant documents are found in the first k positions.

All three metrics are computed at $k \in \{1, 3, 5, 10\}$, averaged over all queries in a NanoBEIR subset, and macro-averaged across the 13 subsets.

2.4.2 Statistical Significance Testing

The retrieval benchmark has a fixed set of queries, and each can be scored by a model. Therefore, model comparison can be thought of as paired tests, which

answer whether a model consistently outperforms another across the full set of queries.

Shapiro–Wilk normality test [30]. This test helps determine which significance test is reliable. It checks to see if the differences $\{d_i\}$ of scores follow a normal distribution.

- H_0: the differences are normally distributed.
- H_1: the differences are not normally distributed.

If H_0 is rejected, the normality assumption of the paired t-test is broken, and the Wilcoxon test (non-parametric test) is the preferred choice.

Paired t-test [32]. A parametric test that assumes the differences are normally distributed and tests whether their mean is zero:

- H_0: $\mu_d = 0$ (the two models have equal mean score).
- H_1: $\mu_d \neq 0$ (one model has a higher mean score).

Wilcoxon signed-rank test [38]. A non-parametric test that does not assume normality and tests whether the median is zero. For each query, the difference d_i is ranked by its magnitude. The test checks to see if the positive ranks (queries where model A wins) always beat the negative ranks (queries where model B wins):

- H_0: the median difference is zero (neither model is better).
- H_1: the median difference is not zero.

Cohen's d [7] quantifies the difference between two models:

$$d = \frac{\bar{d}}{s_d}, \qquad \bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i, \qquad s_d = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(d_i - \bar{d})^2}. \tag{2.14}$$

Values of $|d| \approx 0.2$, 0.5, and 0.8 are conventionally interpreted as small, medium, and large effects, respectively.

2.5 Related Work

The post-hoc methods described in Section 2.3 are already practical at scale. The Hugging Face embedding quantization survey [29] showed that post-hoc binary and INT8 quantization retain strong MTEB quality. This demonstrated that compressed embeddings can be used for large-scale retrieval without retraining. However, it was not clear if training-time methods can do better with the same or less memory.

Bengio et al. [4] came up with the idea for the STE (Section 2.3) for estimating gradients through stochastic neurons. Later, Courbariaux et al. [8] established it as a standard for binarizing neural network weights and activations. The annealed tanh alternative originated in HashNet [5] for deep hashing. Beyond binary quantization, Yang et al. [41] represent a k-level quantizer as a sum of $k-1$ shifted sigmoid functions with learnable biases. Shah et al. [28] reduce STE bias with separate forward and backward temperatures. Schrödter et al. [27] apply a similar sigmoid staircase to input feature compression. None of these multi-bit

or decoupled variants have been applied to retrieval embeddings; they remain a natural direction for future work.

Both STE and annealed tanh have been adopted for dense retrieval. Yamada et al. [40] used annealed tanh in the Binary Passage Retriever (BPR). BPR achieves a 32× memory reduction from 65 GB to 2 GB with zero accuracy loss on Natural Questions and TriviaQA. It uses binary Hamming search for candidate generation, followed by float32 reranking. It was trained with a joint margin ranking and MNRL loss. At production scale, Gan et al. [11] deployed recurrent residual binarization with STE at Tencent. Their approach stacks multiple binary stages to reach a user-specified bit depth at runtime. They report 30–50% index cost savings in web search, video retrieval, and copyright detection. These results suggest that training-time binarization can deliver strong retrieval quality. But every study has a different base model and training data and benchmarking, which makes comparison difficult. STE and annealed tanh have never been compared directly under identical conditions. This thesis isolates that comparison.

MRL (Section 2.3.2.3) is now prominent on the MTEB leaderboard [24] and is a good choice when deployments have different memory budgets. SMEC [45] identifies three problems with standard MRL: gradient fluctuation from optimizing shared encoder weights at the same time from different dimension losses; information loss from blind truncation; and weak contrastive learning from easy in-batch negatives. They solve it by sequential dimension training, adaptive dimension selection, and cross-batch memory. These changes improved the BEIR NDCG@10 score by 1.9 points at 128 dimensions. QAMA [1] combines MRL

with multi-level quantization-aware training. It uses learnable thresholds and a hybrid precision method. Hybrid precision involves using 2 bits for the first 25% of dimensions and 0.5 bits for the last 25%. This gives an effective 1.625 bits per dimension while keeping 95–98% of the full-precision NDCG@10. The method only tests on ModernBERT and MiniLM, so it doesn't show how well it works on other architectures.

CoRECT [6] provides the most comprehensive comparison of post-hoc methods to date. Its primary focus is on how retrieval performance degrades as corpus size scales from 10K to 100M passages. It evaluates eight techniques with four embedding models. It also presents the retrieval performance at various compression ratios for post-hoc methods. They found that quantization performs better than truncation at equal ratios, MRL training matters for truncation quality, and combining quantization with moderate truncation achieves the best scores for most models. CoRECT is the closest prior work in evaluation to this thesis, but it leaves training-time methods entirely out of scope.

No existing study enables a direct comparison between post-hoc and training-time compression under controlled conditions. Most prior studies evaluate a limited set of backbone models, leaving open whether compression trade-offs generalize across architectures of different retrieval starting quality. Stacked combinations involving training-time methods remain unstudied. This thesis addresses all three gaps. In this thesis, all compression variants share the same pre-trained float32 baseline. Post-hoc methods use the fine-tuned float32 model, and training-time methods are fine-tuned with compression awareness on the same corpus and

evaluated on NanoBEIR [44, 34]. These experiments were repeated with three backbones: BGE-base-en-v1.5, RoBERTa-base, and MPNet-base to check the generalizability.

Chapter 3. Datasets

This chapter describes the two categories of data used in this thesis: the multi-source training corpora used to fine-tune the base embedding model, and the evaluation benchmark used to measure retrieval performance. These two categories are kept strictly separate; no training example appears in any evaluation set. Training data is drawn from publicly available question answering, paraphrase, and natural language inference corpora, while evaluation is performed exclusively on the NanoBEIR benchmark.

3.1 Training Data

The embedding model is fine-tuned on a mixture of seven publicly available text retrieval and semantic similarity datasets sourced from the Hugging Face Hub. Drawing from multiple domains and pair types reduces the risk of overfitting to a single retrieval pattern and improves the generalization of learned representations. Each dataset contributes (query, positive passage) pairs or (query, positive passage, negative passage) triplets that are consumed by the Multiple Negatives Ranking Loss during training.

3.1.1 Dataset Sources

The seven training datasets span five broad retrieval and semantic similarity tasks: passage retrieval, title-to-abstract prediction, duplicate question detection, natural language inference, and open-domain question answering. Table 3.1 summarizes each dataset, its pair type, raw size, assigned sampling weight, and the effective number of training examples drawn from it per training epoch under the weighted sampling scheme described in Section 3.1.2.

Table 3.1: Training dataset summary. Effective rows reflect the contribution of each dataset to the training mixture after weighted batch sampling. Total rows across all sources sum to 58.73M; effective rows sum to approximately 4.19M.

S.No.	Dataset	Pair Type	Total Rows	Weight	Effective Rows
1	msmacro_triplet	Query to Passage (triplet)	13.64M	0.15	∼1.94M
2	s2orc_title_abstract	Title to Abstract (pair)	41.77M	0.01	∼0.40M
3	quora_dup_triplet	Duplicate question (triplet)	2.79M	0.15	∼0.40M
4	nli_for_simcse_triplet	Entailment/Contradiction (triplet)	0.27M	2	∼0.52M
5	natural_questions	Question to Answer (pair)	0.10M	4	∼0.38M
6	trivia_qa_triplet	Question to Answer (triplet)	0.06M	4	∼0.23M
7	hotpotqa	Multi-hop Q to A (triplet)	0.08M	4	∼0.32M
Total			**58.73M**		**∼4.19M**

MS MARCO Triplets (*msmacro_triplet*) is derived from the Microsoft MAchine Reading COmprehension dataset [3], the largest source in the mixture with 13.64 million (query, passage, negative passage) triplets. Queries are real user-issued Bing search queries and passages are extracted from web documents.

A low sampling weight of 0.15 is applied to prevent this large corpus from dominating gradient updates relative to the smaller, higher-quality datasets.

Semantic Scholar Open Research Corpus (*s2orc_title_abstract*) provides 41.77 million (title, abstract) pairs drawn from scientific publications across a wide range of academic disciplines [22]. This source trains the model to associate document titles with their corresponding abstracts, which serves as a proxy for document-level retrieval. The very low weight of 0.01 reflects the structural regularity of title-abstract pairs (titles are informationally dense summaries rather than natural-language queries) and prevents this very large scientific-domain corpus from skewing the mixture.

Quora Duplicate Triplets (*quora_dup_triplet*) contains 2.79 million (question, duplicate question, non-duplicate question) triplets drawn from the Quora Question Pairs dataset. This source trains the model to embed semantically equivalent questions in close proximity in representation space, which benefits retrieval tasks where queries may appear in paraphrased form.

NLI for SimCSE Triplets (*nli_for_simcse_triplet*) adapts natural language inference corpora following the SimCSE training recipe [12], yielding 0.27 million (premise, entailment hypothesis, contradiction hypothesis) triplets. Entailment pairs serve as high-quality positives and contradiction pairs serve as hard negatives. Despite its small size, this source receives the second-highest weight of 2, reflecting the quality of its hard negatives and the demonstrated effectiveness of NLI-derived pairs for contrastive representation learning.

Natural Questions (*natural_questions*) contains 0.10 million (question, answer passage) pairs extracted from Google's Natural Questions dataset [19], in which questions are real user queries submitted to the Google search engine and answers are Wikipedia passages identified by human annotators. A high weight of 4 is assigned to compensate for the small size of this dataset relative to the mixture total, and because open-domain question-answering pairs closely match the target retrieval distribution of most benchmark tasks.

TriviaQA Triplets (*trivia_qa_triplet*) provides 0.06 million (trivia question, supporting passage, negative passage) triplets from the TriviaQA reading comprehension dataset [16]. Questions require broad factual knowledge and are paired with supporting evidence passages from Wikipedia and web documents. This source also receives a weight of 4 to counteract its small absolute size.

HotpotQA (*hotpotqa*) contributes 0.08 million multi-hop (question, supporting passage, negative passage) triplets [42]. HotpotQA questions require reasoning across multiple documents to answer, representing a more challenging retrieval scenario than single-hop question answering. A weight of 4 is applied, consistent with the other small question-answering sources.

3.1.2 Weighted Batch Sampling

Rather than concatenating all seven datasets and sampling uniformly, the training pipeline employs weighted batch sampling, where each dataset is assigned a scalar weight that controls the expected number of examples drawn from it per training step. Let w_i denote the weight of dataset i and N_i its total number

of examples. The effective contribution of dataset i to the training mixture per epoch is proportional to $w_i \cdot N_i$, scaled so that the contributions across all datasets sum to approximately 4.19 million effective examples (see Table 3.1).

This scheme serves two purposes. First, it prevents the two largest but more structurally constrained sources (MS MARCO and S2ORC, which together account for 94 percent of raw examples) from overwhelming the gradient signal with a single retrieval pattern. Second, it allows smaller domain-specific question-answering datasets (Natural Questions, TriviaQA, and HotpotQA) to contribute meaningfully to training despite their limited raw size. The resulting effective training mixture of approximately 4.19 million examples per epoch is substantially smaller than the 58.73 million raw examples, which reduces wall-clock training time while preserving the diversity of the training signal across retrieval tasks and domains.

3.2 Evaluation Benchmark

All experiments in this thesis are evaluated on the NanoBEIR benchmark. The full BEIR benchmark, from which NanoBEIR is derived, was considered but not used due to the prohibitive cost of encoding corpora containing millions of passages for every compression variant under study.

3.2.1 NanoBEIR

NanoBEIR [44] is a lightweight subset of the Benchmarking Information Retrieval (BEIR) benchmark [34], constructed by subsampling each BEIR dataset

to 50 evaluation queries and a small accompanying corpus. It is distributed on the Hugging Face Hub as part of the *sentence-transformers* ecosystem and is designed to enable rapid model comparisons without the computational cost of encoding the full BEIR corpora, making it well suited for iterative development and ablation studies.

This thesis uses all 13 available NanoBEIR subsets, summarized in Table 3.2. The subsets span a diverse set of retrieval tasks, including fact verification, question answering, entity retrieval, citation prediction, biomedical search, and argument retrieval, and cover domains ranging from general web content to financial documents to scientific literature. Each subset provides 50 queries (49 in the case of NanoTouche2020) and a corpus ranging from 2,260 to 6,100 documents, for a total of 649 queries and 57,390 corpus documents across all 13 subsets.

The diversity of NanoBEIR tasks is particularly important for evaluating compression methods. A compression technique that degrades performance on some tasks while improving or maintaining it on others would produce a misleading average score if evaluated on a homogeneous benchmark. By spanning 13 qualitatively different retrieval scenarios, NanoBEIR provides a more robust signal for assessing whether a compression method is broadly applicable.

Table 3.2: NanoBEIR benchmark subsets used for evaluation. All subsets contain 50 queries except NanoTouche2020, which contains 49. Corpus sizes range from 2,260 to 6,100 documents. The total across all subsets is 649 queries and 57,390 corpus documents.

S.No.	Subset	Queries	Corpus	Task (Query Type to Document Type)
1	NanoArguAna	50	3,690	Counter-argument retrieval (Claim to Argument)
2	NanoClimateFEVER	50	3,460	Fact-checking (Claim to Document)
3	NanoDBPedia	50	6,100	Entity retrieval (Query to Document)
4	NanoFEVER	50	5,050	Fact-checking (Claim to Document)
5	NanoFiQA2018	50	4,650	Financial question answering (Question to Document)
6	NanoHotpotQA	50	5,140	Multi-hop question answering (Question to Document)
7	NanoMSMARCO	50	5,090	Web search / general QA (Question to Passage)
8	NanoNFCorpus	50	3,000	Biomedical information retrieval (Query to Document)
9	NanoNQ	50	5,090	Open-domain question answering (Question to Document)
10	NanoQuoraRetrieval	50	5,100	Duplicate detection (Question to Question)
11	NanoSCIDOCS	50	2,260	Citation prediction (Title to Document)
12	NanoSciFact	50	2,970	Scientific fact-checking (Claim to Document)
13	NanoTouche2020	49	5,790	Conversational argument retrieval (Question to Argument)
Total		**649**	**57,390**	

Chapter 4. Methodology

4.1 Experimental Design

The goal of this thesis is to compare embedding compression methods on equal terms. Prior work makes this hard. Different papers use different backbones, different training data, and different benchmarks (Section 2.5). This thesis controls all three.

Every method studied here is applied to the same set of backbone models. Every backbone is fine-tuned under the same conditions. Every variant is evaluated on the same benchmark with the same pipeline.

The experiments are organized into two tracks, illustrated in Figure 4.1. Both start from the same pre-trained backbone weights.

4.1.0.0.1 Post-hoc track. Each backbone is fine-tuned without any compression, producing a fine-tuned fp32 baseline checkpoint. Compression is then applied to the embeddings produced by this checkpoint; the checkpoint itself is not touched by the compression step.

4.1.0.0.2 Training-time track. Each backbone is fine-tuned with a modified architecture, a modified loss, or both. This track does not start from the fine-

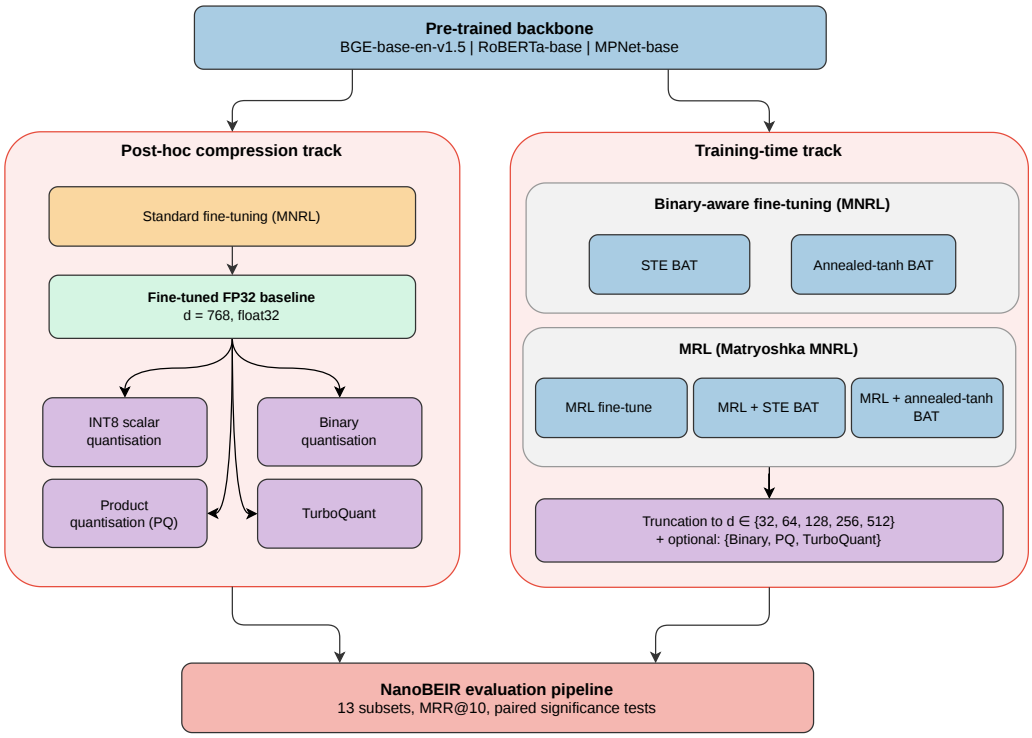

Figure 4.1: Overview of the experimental methodology. Each pre-trained backbone (BGE-base-en-v1.5, RoBERTa-base, MPNet-base) is taken through two independent tracks. The post-hoc compression track fine-tunes a standard fp32 baseline with MNRL and then applies INT8, binary, PQ, or TurboQuant compression to the cached embeddings. The training-time track modifies the architecture or loss during fine-tuning: STE or annealed-tanh binarization-aware training (BAT), Matryoshka Representation Learning (MRL), and joint MRL+BAT variants. MRL checkpoints additionally support post-hoc truncation to $d \in \{32, 64, 128, 256, 512\}$, optionally stacked with binary, PQ, or TurboQuant. All resulting variants are evaluated on the NanoBEIR benchmark with paired significance tests.

tuned fp32 baseline. It starts from the pre-trained backbone weights, the same as the post-hoc track.

All variants are evaluated on NanoBEIR using the pipeline described in Section 4.6. Table 4.1 lists the six training variants and the post-hoc transforms applied to each.

Table 4.1: Summary of training variants. Each variant is a complete fine-tuning run from a pre-trained backbone. Post-hoc transforms are applied after training with no weight modification.

Variant	**Architecture change**	**Loss function**	**Post-hoc options**
Fine-tuned fp32 baseline	None	MNRL	INT8, Binary, PQ, TurboQuant
STE binarization-aware	+ BinarizationLayer (STE)	MNRL	Binary
MRL	None	MatryoshkaLoss (MNRL)	Truncation; Truncation + INT8/Binary
MRL + STE	+ BinarizationLayer (STE)	MatryoshkaLoss (MNRL)	Truncation; Truncation + Binary
Annealed tanh binarization-aware	+ AnnealedTanhLayer	MNRL	Binary
MRL + Annealed tanh	+ AnnealedTanhLayer	MatryoshkaLoss (MNRL)	Truncation; Truncation + Binary

4.2 Training Setup

All training variants are run independently on three backbone architectures: BGE-base-en-v1.5, RoBERTa-base, and MPNet-base (Section 2.1). No weights are shared across backbones.

The training objective is the Multiple Negatives Ranking Loss (MNRL, Section 2.2) computed with cosine similarity, with in-batch negatives drawn from the other positive passages in the same batch. Batches are sampled from the eight-dataset training mixture (Chapter 3) using a weighted sampler that controls each dataset's contribution per epoch.

The fine-tuned fp32 baseline is produced by this setup with no additional compression layer. It serves as the starting point for all post-hoc compression methods. Training-time variants use the same setup, with modifications to the architecture, the loss, or both (Section 4.3); they fine-tune from the pre-trained backbone weights, not from the baseline checkpoint.

Hyperparameters are shared across all variants and backbones; none are tuned per-variant. Table 4.2 summarises the configuration.

Table 4.2: Training hyperparameters shared across all experiments and backbone models. Effective batch size accounts for per-device batch, number of GPUs, and gradient accumulation.

Hyperparameter	**Value**
Optimizer	AdamW
Learning rate	2×10^{-5}
LR scheduler	Linear decay with warmup
Warmup ratio	0.10
Weight decay	0.10
Per-device batch size	16
Gradient accumulation steps	8
Effective batch size	512 (16×4 GPUs $\times$ 8 steps)
Training epochs	1
Max sequence length	512 tokens
Precision	bfloat16

4.3 Training-Time Compression Experiments

Each variant below modifies the architecture, the loss, or both. All other settings follow Section 4.2.

4.3.1 STE Binarization-Aware Training

A binarization layer is appended after mean pooling. The forward pass applies sign$(\cdot)$ to the pooled embedding, mapping each dimension to $\{-1, +1\}$; the backward pass uses the Straight-Through Estimator (Section 2.3), passing gradients to the encoder unchanged. The training loss is MNRL with cosine similarity, computed on the binarized outputs. Training encourages the encoder to produce pre-binarization activations already close to ± 1, minimising the information loss of the sign step at inference.

4.3.2 MRL Fine-Tuning

The model is identical to the baseline; only the loss changes. MNRL is wrapped by the Matryoshka loss (Section 2.3), which evaluates it at six nested granularities $\{32, 64, 128, 256, 512, 768\}$. The five truncation granularities $\{32, 64, 128, 256, 512\}$ are the supported post-hoc truncation points; the full-dimension loss ($d = 768$) prevents degradation of the complete representation. All six losses are weighted equally ($c_m = 1$, Equation 2.9). The trained checkpoint is structurally identical to the baseline; truncation is applied post-hoc by the evaluation pipeline (Section 5.4).

4.3.3 Joint MRL and STE Binarization-Aware Training

This variant applies both the binarization layer of Section 4.3.1 and the Matryoshka loss of Section 4.3.2, training representations that are simultaneously nested and binarization-aware.

4.3.4 Annealed Tanh Binarization-Aware Training

The binarization layer of Section 4.3.1 is replaced with the annealed-tanh surrogate of Section 2.3. The annealing rate is $\gamma = 0.1$, following [40]; the empirical validation across $\gamma \in \{0.05, 0.1, 0.2\}$ is reported in Table 1. After approximately $t = 8{,}000$ steps (one epoch over the training mixture), $\beta \approx 28$, at which point $\tanh(\beta_t \cdot \mathbf{e}) \approx \text{sign}(\mathbf{e})$ for typical activation magnitudes. At inference the layer switches to $\text{sign}(\cdot)$. The loss is MNRL with cosine similarity applied to the soft-binarized embeddings during training.

4.3.5 Joint MRL and Annealed Tanh Binarization-Aware Training

This variant combines the annealed-tanh layer of Section 4.3.4 with the Matryoshka loss of Section 4.3.2.

4.4 Post-Hoc Compression

Post-hoc methods are applied to the cached embeddings of the fine-tuned fp32 baseline; the model weights are not modified.

4.4.1 INT8 Scalar Quantization

Following Section 2.3, calibration bounds are derived from the first 1,000 corpus embeddings (after any dimensionality truncation) and applied identically to corpus and query embeddings. At retrieval time, the int8 values are scored directly with cosine similarity (cast to float for normalisation, without an explicit dequantisation step; since cosine is scale-invariant, this yields the same ranking as scoring the dequantised vectors).

4.4.2 Binary Quantization

Post-hoc binary quantization follows the procedure in Section 2.3, with bits packed into `uint8`. Similarity is Hamming similarity between packed binary query and corpus vectors.

4.4.3 Product Quantization

PQ is fitted on the corpus embeddings using Faiss [10]. Sub-space counts $M \in \{16, 24, 32, 48, 64, 96, 128, 192\}$ are evaluated with centroid bit-widths $n_{\text{bits}} \in \{4, 8\}$.

4.4.4 TurboQuant

TurboQuant [43] is calibrated on the corpus embeddings. Bit-depths of $\{1, 2, 3, 4, 8\}$ bits per dimension are evaluated.

4.5 Stacked Combinations

MRL-trained checkpoints support simultaneous compression along two dimensions: dimensionality reduction via post-hoc truncation, followed by a precision-reducing transform. Each MRL checkpoint is truncated to each of the five granularities $\{32, 64, 128, 256, 512\}$, and then binary quantization, PQ, or TurboQuant is applied to the truncated embedding. This yields the grid of (backbone $\times$ truncation dimension $\times$ precision method) combinations evaluated in Chapter 5.

4.6 Evaluation Framework

4.6.1 Embedding Cache and Transform Pipeline

Each checkpoint encodes the NanoBEIR corpus and queries once into float32 embeddings, which are cached on disk. All compression variants derived from a given checkpoint are then evaluated by applying their transforms to these cached embeddings, with no re-encoding.

A variant applies at most one optional dimensionality truncation, followed by at most one precision-reducing transform. The transforms are applied in this fixed order:

1. Dimensionality truncation (MRL variants only).
2. Binary quantization with `uint8` packing.
3. INT8 scalar quantization.
4. Product quantization (PQ).

5. TurboQuant.

Steps 2–5 are mutually exclusive: only one precision-reducing transform is active per variant. Step 1 may combine with any of steps 2–5, producing the stacked combinations defined in Section 5.4. The hyperparameters for each transform (PQ sub-spaces M and bits per centroid; TurboQuant bits per dimension; INT8 calibration bounds) are specified in Section 4.4.

4.6.2 Metrics

The three retrieval metrics used in this thesis, NDCG@k, MRR@k, and Recall@k, are defined in Section 2.4. All are computed at $k \in \{1, 3, 5, 10\}$, averaged over the 50 queries per NanoBEIR subset, then macro-averaged across the 13 subsets. MRR is the primary reported metric throughout Chapter 5.

4.6.3 Memory Budget and Compression Ratio

Embedding memory is computed as:

$$\text{memory (bytes)} = n \times d_{\text{stored}} \times \frac{b}{8}, \tag{4.1}$$

where n is the number of embeddings, d_{stored} is the stored dimension (number of PQ sub-spaces for PQ variants), and b is the number of bits per stored element. For float32, $b = 32$; for INT8, $b = 8$; for binary packed embeddings, $b = 1$.

The compression ratio r is reported relative to the float32 baseline at the original embedding dimension. A 768-dimensional float32 embedding occupies

$768 \times 4 = 3{,}072$ bytes per vector. The ratio takes the following form for each method family:

- Scalar precision reduction to b bits per component: $r = 32/b$ (INT8: $4\times$; binary: $32\times$).
- MRL truncation to d' dimensions: $r = 768/d'$.
- Product quantization with M sub-spaces and n-bit codes per sub-space: $r = (768 \times 32)/(M \times n)$.
- TurboQuant at b bits per dimension: $r = 32/b$.

Stacked combinations multiply the per-axis ratios. For example, MRL-256 followed by binary quantization gives $(768/256) \times 32 = 96\times$ compression.

4.6.4 Statistical Significance Testing

The tests used here are defined in Section 2.4.2. For each model pair, per-query scores are collected across all 13 NanoBEIR subsets and pooled into a single paired sample (649 query pairs per comparison). The per-query difference is $d_i = s_A(q_i) - s_B(q_i)$ for each query q_i, where $s(\cdot)$ denotes the retrieval score of the respective model. The Shapiro–Wilk test is first applied to this vector of differences to assess normality. If normality is not rejected, the paired t-test is used; otherwise the Wilcoxon signed-rank test is used. Cohen's d is reported as an effect size in either case. Results in Chapter 5 are flagged as statistically significant at $\alpha = 0.05$.

Chapter 5. Results and Analysis

All results in this chapter follow the experimental design and evaluation framework set out in Chapter 4. Unless stated otherwise, the primary reported metric is mean MRR@10 macro-averaged across the 13 NanoBEIR subsets (Section 4.6.2), and compression ratios are reported relative to the float32 baseline at the original embedding dimension (Section 4.6.3).

5.1 Float32 Baselines

Table 5.1 reports mean MRR@10 macro-averaged across all 13 NanoBEIR subsets for each backbone in its pre-trained and fine-tuned float32 configuration. These results serve as the upper-bound reference for all compression experiments in Sections 5.2–5.5.

Table 5.1: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for pre-trained and fine-tuned float32 embeddings. Δ is the absolute change due to fine-tuning.

Configuration	**BGE**	**RoBERTa**	**MPNet**
Pre-trained	0.691	0.097	0.156
Fine-tuned	0.685	0.530	0.582
Δ	**−0.006**	**+0.434**	**+0.426**

Pre-trained RoBERTa and MPNet score 0.097 and 0.156 MRR@10. MNRL fine-tuning lifts them to 0.530 and 0.582, gains of 43.4 and 42.6 percentage points (pp). BGE is essentially unchanged: the 0.6 pp delta (0.691 $\rightarrow$ 0.685) is not statistically significant (paired Wilcoxon signed-rank test on per-query MRR@10, $p = 0.49$; Appendix B.3). It has already been pre-trained for retrieval at large scale, so further adaptation on our smaller mixed-domain corpus adds no benefit. Unless stated otherwise, all later compression results are reported relative to the fine-tuned float32 baseline for each backbone.

Even after fine-tuning, BGE leads MPNet by 6.6 percentage points and RoBERTa by 15.5 percentage points. Whether compression methods preserve this ordering across backbones is examined in Section 5.6.

5.2 Post-Hoc Compression

This section evaluates compression applied after training to the fine-tuned float32 checkpoints, with no modification to model weights. Four methods are examined in increasing order of compression depth: INT8 scalar quantisation (Section 5.2.1), post-hoc binary quantisation (Section 5.2.2), Product Quantisation (Section 5.2.3), and TurboQuant (Section 5.2.4). Section 5.2.5 places all four methods on a common axis at 32$\times$ compression.

5.2.1 INT8 Scalar Quantisation

INT8 is essentially lossless at 4× compression across all three backbones (Table 5.2): the largest delta from float32 is −1.5 percentage points (MPNet), and RoBERTa is within noise of its float32 baseline.

Table 5.2: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for float32 and INT8 (4× compression). Δ is the absolute change due to INT8 quantisation.

Configuration	**BGE**	**RoBERTa**	**MPNet**
Float32 (fine-tuned)	0.685	0.530	0.582
INT8 (4×)	0.678	0.532	0.567
Δ	**−0.007**	**+0.002**	**−0.015**

5.2.2 Binary Quantisation

Table 5.3: Mean MRR@10 for post-hoc binary quantisation (32×) compared with the fine-tuned float32 baseline.

Configuration	**BGE**	**RoBERTa**	**MPNet**
Float32 (fine-tuned)	0.685	0.530	0.582
Binary (32×)	0.646	0.501	0.534
Δ	**−0.039**	**−0.029**	**−0.048**

At 32× compression, post-hoc binary causes a meaningful quality drop across all backbones (Table 5.3): −4.8 pp for MPNet, −3.9 pp for BGE, −2.9

pp for RoBERTa. Section 5.3 shows that training-time binarisation substantially recovers this gap.

5.2.3 Product Quantisation

Thirteen PQ configurations are evaluated, spanning 32× to 384× compression and covering both $n = 4$ and $n = 8$ bits per subspace at every shared memory budget. Figure 5.1 shows the resulting quality–compression curves.

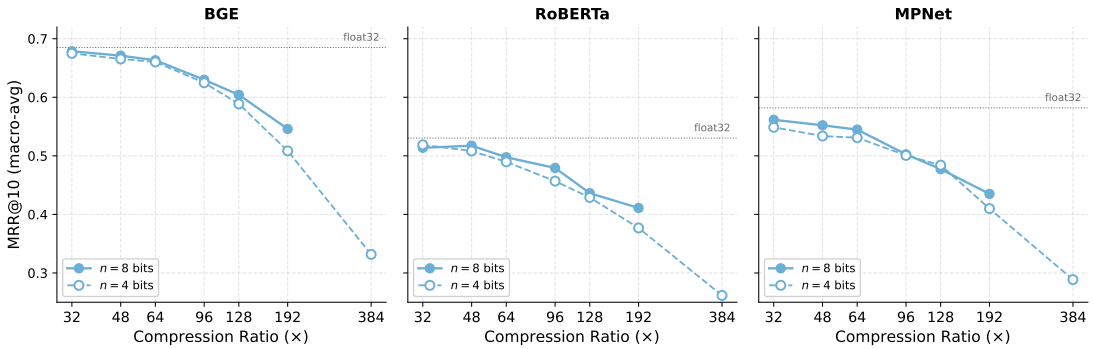

Figure 5.1: Mean MRR@10 (macro-averaged over the 13 NanoBEIR subsets) for Product Quantisation as a function of compression ratio, one panel per backbone. Solid lines with filled markers use $n = 8$ bits per subspace; dashed lines with hollow markers use $n = 4$. Each point on a curve corresponds to a different number of subspaces M. The dotted horizontal line marks the fine-tuned float32 baseline for each backbone.

Two findings stand out. First, PQ is near-lossless at 32× on BGE: the $n = 8$ configuration scores 0.679 against a float32 ceiling of 0.685. Quality degrades gracefully through 96× and then collapses at 384×, where the codebook size of $2^4 = 16$ centroids per subspace becomes the binding constraint. Second, at any fixed compression ratio the $n = 8$ variant beats $n = 4$ in 16 of 18 backbone-ratio pairs; the advantage is largest at 192× (up to +3.7 percentage points) and shrinks

toward zero by 32×. Finer codebook resolution within a subspace recovers more retrieval signal than splitting the embedding into more, coarser subspaces.

5.2.4 TurboQuant

Five TurboQuant (TQ) configurations are evaluated, spanning 4× to 32× compression (8, 4, 3, 2, and 1 bits per embedding dimension). Figure 5.2 shows the quality–compression curves and Table 5.4 reports the numbers.

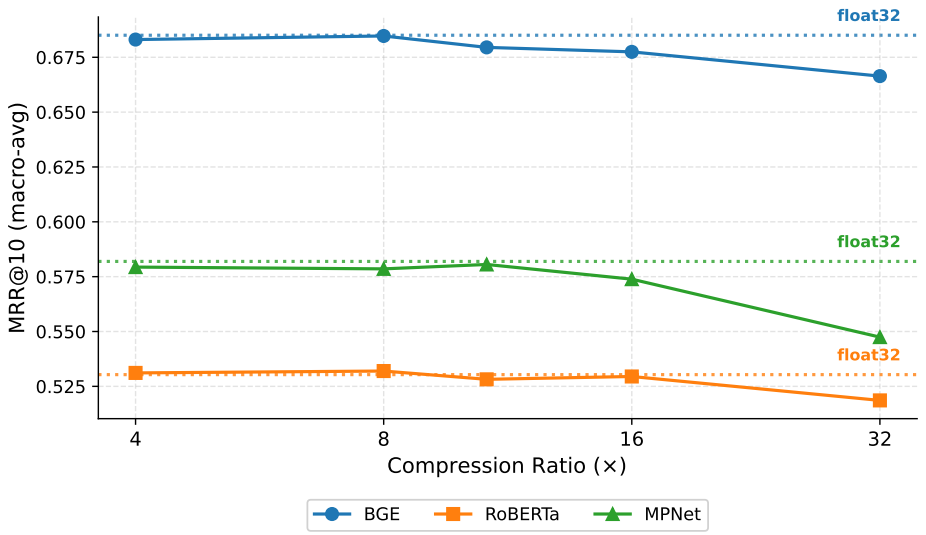

Figure 5.2: Mean MRR@10 (macro-averaged over the 13 NanoBEIR subsets) for TurboQuant as a function of compression ratio, one line per backbone. Each point corresponds to a bit-width $b \in \{1, 2, 3, 4, 8\}$. The dotted horizontal lines mark the fine-tuned float32 baseline for each backbone.

TQ-4bit (8×) is near-lossless on all backbones: BGE matches its float32 ceiling (0.685 vs 0.685), and the curve plateaus between 4-bit and 8-bit, so 4 bits per dimension is the operating-point sweet spot. Degradation to 32× (1-bit) is modest: BGE loses 1.9 pp, RoBERTa 1.1 pp, and MPNet 3.5 pp from float32.

Table 5.4: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for TurboQuant across the five evaluated bit-widths, with the fine-tuned float32 baseline for reference.

Configuration	**Ratio**	**BGE**	**RoBERTa**	**MPNet**
Float32 (fine-tuned)	1×	0.685	0.530	0.582
TQ-8bit	4×	0.683	0.531	0.579
TQ-4bit	8×	0.685	0.532	0.579
TQ-3bit	10.7×	0.680	0.528	0.581
TQ-2bit	16×	0.677	0.529	0.574
TQ-1bit	32×	0.666	0.519	0.547

5.2.5 Cross-Method Comparison at 32×

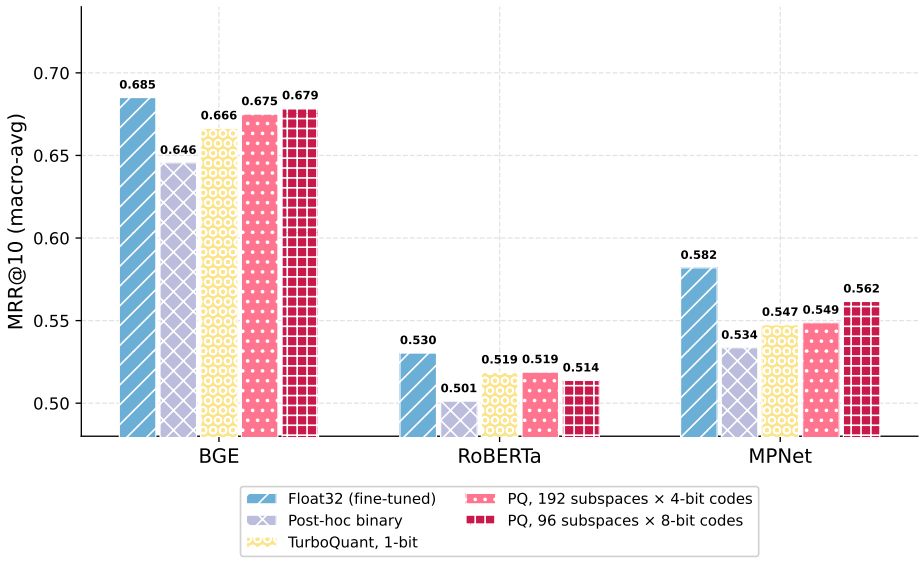

Figure 5.3: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for the fine-tuned float32 baseline and the four post-hoc methods at 32× compression, grouped by backbone. Y-axis starts at 0.48 to highlight inter-method differences.

Table 5.5: Mean MRR@10 (macro-averaged over 13 NanoBEIR subsets) for the four post-hoc methods at 32× compression, with the fine-tuned float32 baseline for reference. Best per backbone in bold; ties at displayed precision are bolded together.

Method	**Ratio**	**BGE**	**RoBERTa**	**MPNet**
Float32 (fine-tuned)	1×	0.685	0.530	0.582
Post-hoc binary	32×	0.646	0.501	0.534
TurboQuant, 1-bit	32×	0.666	**0.519**	0.547
PQ, 192 subspaces × 4-bit codes	32×	0.675	**0.519**	0.549
PQ, 96 subspaces × 8-bit codes	32×	**0.679**	0.514	**0.562**

At 32× compression the best method is backbone-dependent (Figure 5.3, Table 5.5): PQ with 8-bit codes (96 subspaces) wins for BGE and MPNet, while TurboQuant 1-bit and PQ with 4-bit codes (192 subspaces) are essentially tied for RoBERTa. Post-hoc binary is the weakest method across all three backbones. Training-time methods (Section 5.3) offer a complementary path to closing the remaining gap to float32.

5.3 Training-Time Compression

Unlike post-hoc methods, training-time compression modifies the fine-tuning procedure itself so that the model learns representations optimised for the compressed form. Two families are evaluated: binarisation-aware training (Section 5.3.1), which targets the same 32× compression ratio as post-hoc binary quantisation, and Matryoshka Representation Learning (Section 5.3.2), which targets dimensionality reduction.

5.3.1 Binarisation-Aware Training: STE and Annealed Tanh

Two binarisation-aware training (BAT) variants are evaluated, differing only in the gradient surrogate used for $\text{sign}(x)$ (Section 4.3): BAT-STE (Straight-Through Estimator) and BAT-atanh (annealed tanh at $\gamma = 0.1$, the principled default following [40]; the full ablation over $\gamma \in \{0.05, 0.1, 0.2\}$ is in Appendix A.1). Both produce 768-dimensional binary embeddings at inference (32× compression, 96 bytes per vector).

Table 5.6: Mean MRR@10 at 32× compression for binarisation-aware training methods, post-hoc binary, and the fine-tuned float32 baseline (ceiling). Results are macro-averaged across the 13 NanoBEIR subsets. Best 32× result per backbone in bold. The annealed-tanh annealing rate is fixed at $\gamma = 0.1$ throughout; the full ablation over γ is reported in Appendix A.1.

Method	Ratio	BGE	RoBERTa	MPNet
Float32 (fine-tuned, ceiling)	1×	0.685	0.530	0.582
Post-hoc binary	32×	0.645	0.501	0.534
BAT-STE	32×	**0.658**	0.565	**0.582**
BAT-atanh ($\gamma = 0.1$)	32×	0.638	**0.590**	0.580

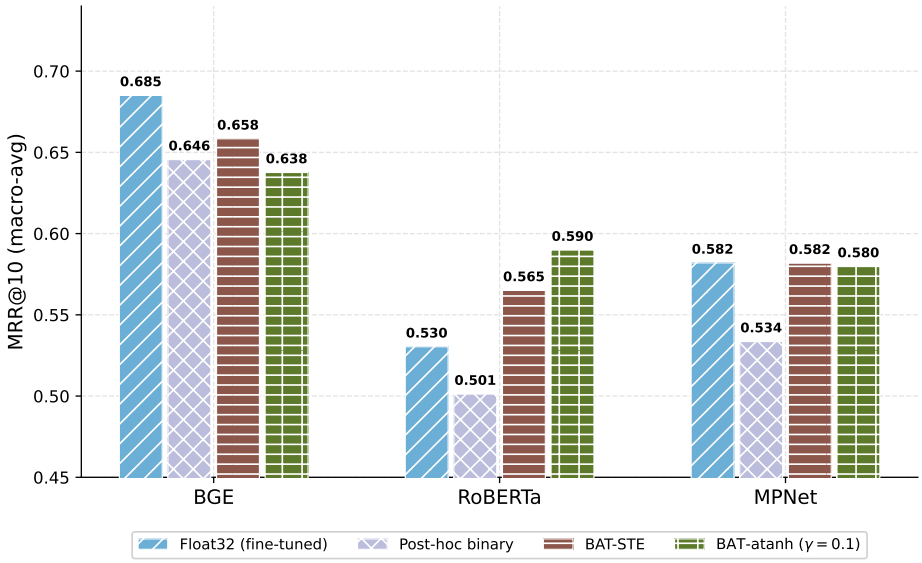

Figure 5.4: Mean MRR@10 at 32× compression for the fine-tuned float32 baseline, post-hoc binary, BAT-STE, and BAT-atanh (γ=0.1) across the three backbones. The full ablation over γ is reported in Appendix A.1.

Both BAT variants substantially exceed post-hoc binary at 32× for all backbones (Table 5.6, Figure 5.4). BAT-STE gains 1.3, 6.4, and 4.8 percentage points (pp) over post-hoc binary on BGE, RoBERTa, and MPNet respectively, with the largest gains on the general-purpose backbones whose float32 geometry is least adapted to retrieval. BAT-STE matches the fine-tuned float32 ceiling on MPNet (0.582) and exceeds it by 3.5 pp on RoBERTa (0.565 vs 0.530; paired Wilcoxon signed-rank test on per-query MRR@10, $p = 0.005$; Appendix B.4), indicating that the binarisation constraint can shape a more retrieval-effective geometry than unconstrained fine-tuning when the starting checkpoint is not already retrieval-specialised.

The two surrogates show a backbone-dependent trade-off. BAT-atanh wins on RoBERTa (0.590 vs 0.565, +2.5 pp), the strongest binary result for RoBERTa across all methods and significantly above the float32 fine-tune (0.590 vs 0.530; paired Wilcoxon $p < 10^{-3}$; Appendix B.4). BAT-STE wins on BGE (+2.0 pp) and effectively ties on MPNet (+0.2 pp). The ablation in Appendix A.1 shows the best γ differs by backbone, but no backbone moves by more than 0.014 MRR@10 across $\gamma \in \{0.05, 0.1, 0.2\}$, so the choice has little practical effect.

5.3.2 Matryoshka Representation Learning

Matryoshka Representation Learning (MRL, Section 4.3.2) trains a single 768-dimensional embedding whose leading d dimensions remain independently informative, so deployment-time truncation to $d \in \{32, 64, 128, 256, 512\}$ requires no retraining.

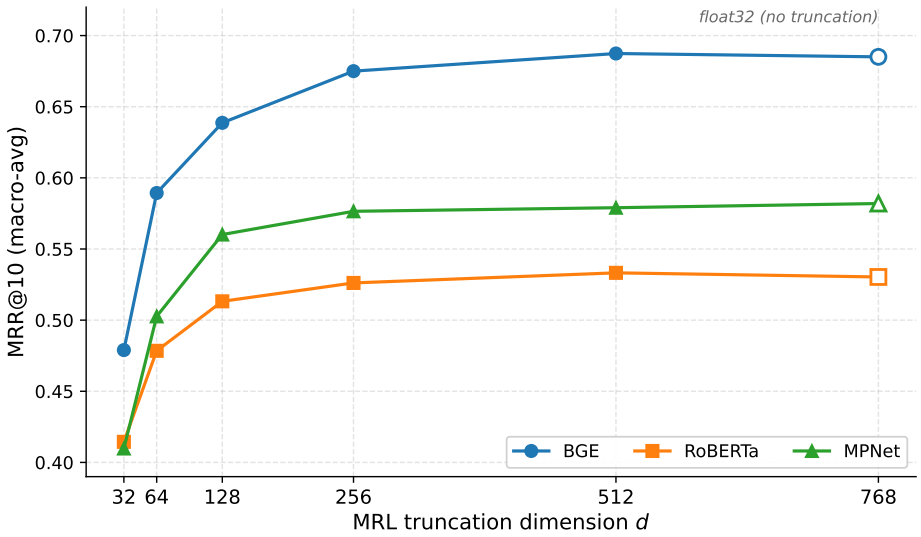

Figure 5.5: Mean MRR@10 as a function of MRL truncation dimension d for all three backbones. The right-most point at $d = 768$ is the fine-tuned float32 baseline (no truncation), shown with hollow markers.

Table 5.7: Mean MRR@10 for MRL truncations across five dimensions, with the 768-dimensional float32 baseline for reference. Compression ratio is $768/d$.

Dim	**Ratio**	**BGE**	**RoBERTa**	**MPNet**
768 (float32)	1×	0.685	0.530	0.582
512	1.5×	0.687	0.533	0.579
256	3×	0.675	0.526	0.576
128	6×	0.639	0.513	0.560
64	12×	0.589	0.478	0.503
32	24×	0.479	0.414	0.410

Figure 5.5 and Table 5.7 show smooth quality degradation as d decreases. At d=512 (1.5× compression), all three backbones match their float32 baselines to within 0.3 percentage points, with BGE (+0.2 pp) and RoBERTa (+0.3 pp) marginally exceeding the full 768-dimensional checkpoint. At d=256 (3×), losses remain within 1.0 pp for every backbone (BGE −1.0 pp, RoBERTa −0.4 pp, MPNet −0.5 pp). Degradation accelerates below d=128: at d=32 (24×), BGE

loses 20.6 pp, MPNet 17.2 pp, and RoBERTa 11.6 pp. The primary value of MRL is its compatibility with precision-reduction methods; the stacked combinations are evaluated in Section 5.4.

5.4 Stacked Combinations

Stacked combinations pair MRL truncation with a second compression step. Two taxonomic groups are evaluated. *Pure-training-time stacks* (MRL+BAT-STE, MRL+BAT-atanh) bake both objectives into the loss, so the binarisation surrogate co-trains with the MRL nested-truncation structure. *Hybrid stacks* (MRL+post-binary, MRL+TQ, MRL+PQ) apply a post-hoc compression step to an MRL-trained checkpoint. Figure 5.6 plots the best-quality configuration of each method at every represented compression ratio across the three backbones.

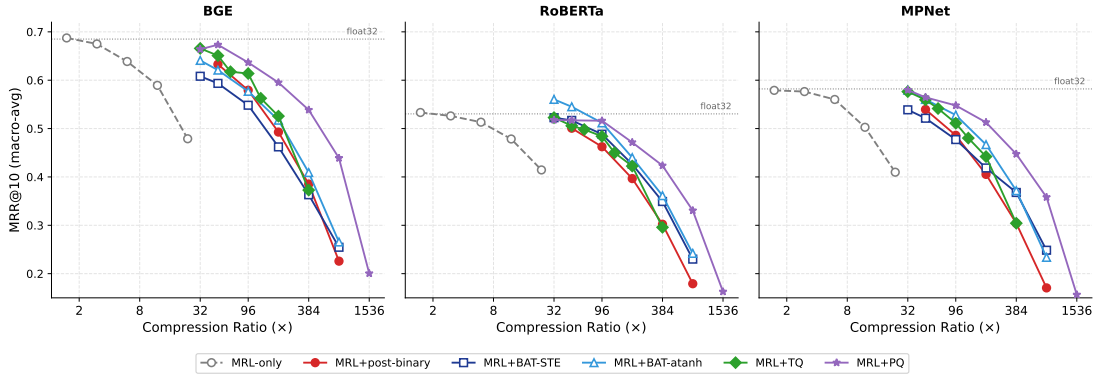

Figure 5.6: Mean MRR@10 vs compression ratio for the five stacked combinations and MRL-only (grey dashed reference) across the three backbones. Each line is the best-quality configuration of that method at every represented ratio. Hollow markers denote pure-training-time stacks (MRL+BAT-STE, MRL+BAT-atanh); filled markers denote hybrid stacks (MRL+post-binary, MRL+TQ, MRL+PQ). The fine-tuned float32 baseline is shown as a horizontal dotted line per panel.

Four findings read off Figure 5.6.

5.4.0.0.1 MRL+PQ dominates the high-compression regime. At every ratio $\geq 96\times$ on every backbone, MRL+PQ is the single best stacked method, and the margin over the second-best widens with compression. At 192×, MRL+PQ leads by 6.9 pp on BGE, 3.1 pp on RoBERTa, and 4.6 pp on MPNet; at 384× those gaps grow to 13.0, 6.2, and 7.5 pp.

5.4.0.0.2 MRL+BAT-atanh exceeds the float32 baseline for RoBERTa. At 32×, MRL+BAT-atanh reaches 0.560 against a fine-tuned 768-dim float32 baseline of 0.530 (+3.0 pp). This mirrors the standalone result of Section 5.3.1, where BAT-atanh on RoBERTa exceeds float32 by +6.0 pp at 32×; the benefit survives MRL truncation. RoBERTa is the only backbone on which any configuration in this section crosses its own float32 ceiling.

5.4.0.0.3 MRL+BAT-atanh strictly dominates MRL+BAT-STE. Across 32–384× on every backbone, MRL+BAT-atanh exceeds MRL+BAT-STE by 1–6 pp (at 96×: BGE +2.9, RoBERTa +2.4, MPNet +5.1 pp). The straight-through gradient forces sign separability from the first optimiser step, conflicting with MRL's requirement that the leading sub-dimensions remain informative as float32 magnitudes across five nested budgets; BAT-atanh tightens the same constraint only gradually, leaving the early-training geometry close to the standard MRL optimum.

5.4.0.0.4 Stacking pushes the quality–compression frontier outward.

The MRL-only reference (grey dashed) in Figure 5.6 stops at 24× (d=32); dimensionality reduction alone cannot compress further. Every stacked curve extends far past this ceiling. MRL+PQ in particular retains meaningful quality through 768×–1536× on all three backbones, well beyond both the MRL-only endpoint and the 384× collapse point of standalone PQ (Section 5.2.3). Combining the two axes reaches operating regimes that neither single-axis method can serve.

The Pareto consequences of these findings are followed up in Section 5.5.

5.5 Pareto Analysis

A configuration is *Pareto-optimal* if no other configuration beats it on both retrieval quality and compression ratio at the same time. The set of all such configurations forms the *Pareto frontier*: the menu of best available trade-offs. Any configuration that lies off the frontier is strictly worse than some point on it and should not be chosen.

This section reports the frontier for each backbone (Section 5.5.1). The split between training-time and post-hoc methods on the frontier is then analysed across backbones in Section 5.6.

5.5.1 Pareto Frontiers

5.5.1.0.1 BGE frontier.

The BGE frontier (Figure 5.7, Table 5.8) covers four orders of magnitude of compression. Table 5.8 lists the Pareto-optimal con-

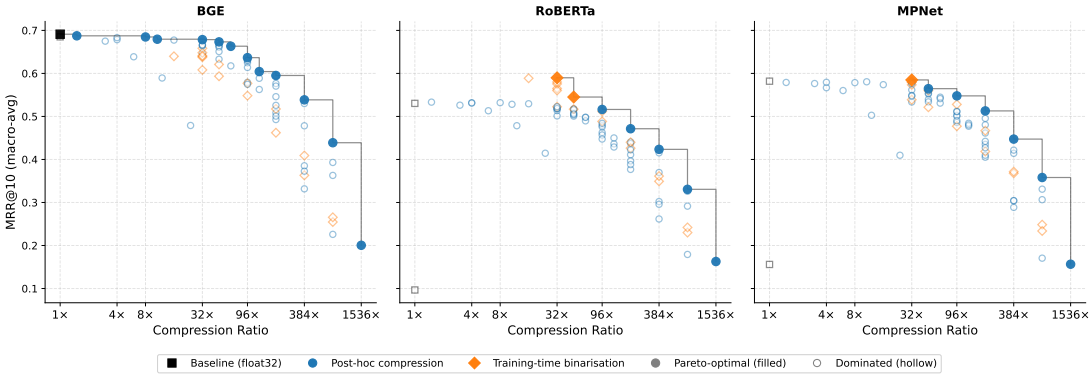

Figure 5.7: Quality vs compression Pareto frontiers for BGE, RoBERTa, and MPNet. The x-axis is on a log scale. Filled markers are Pareto-optimal; hollow markers are dominated. Colour encodes method family: post-hoc (blue) vs training-time binarisation (orange); float32 baselines are black.

figurations from 1× up to 96×; the frontier extends through stacked MRL+PQ configurations to 1536× (MRR@10 = 0.200), visible in Figure 5.7.

Table 5.8: BGE Pareto-optimal configurations, ordered by compression ratio. Method family is indicated for each point.

Method	**Family**	**Compression Ratio**	**Bytes**	**MRR@10**
Pre-trained BGE	Baseline	1×	3072	0.691
MRL-512	Post-hoc	1.5×	2048	0.687
TQ-4bit	Post-hoc	8×	384	0.685
TQ-3bit	Post-hoc	10.7×	288	0.680
PQ (M=96, n=8)	Post-hoc	32×	96	0.679
MRL-256 + PQ (M=64, n=8)	Post-hoc	48×	64	0.673
PQ (M=48, n=8)	Post-hoc	64×	48	0.663
MRL-256 + PQ (M=32, n=8)	Post-hoc	96×	32	0.637

Three things stand out from this frontier. The 1× point is the pre-trained checkpoint, not the fine-tuned one, because fine-tuning slightly reduces BGE qual-

ity (Section 5.1). TurboQuant dominates the moderate range (8× to 10.7×); moving from 1.5× to 8× costs only 0.2 pp of MRR@10. From 32× onwards PQ takes over: PQ (M=96, n=8) at 32× beats both TQ-1bit and BAT-STE, and stacked MRL+PQ configurations extend the frontier all the way to 1536×.

5.5.1.0.2 RoBERTa and MPNet frontiers. RoBERTa and MPNet differ from BGE in one important way: their 32× frontier point is a training-time binarisation method, not PQ.

- RoBERTa at 32×: BAT-atanh (γ=0.1) scores 0.590, beating the best post-hoc method (PQ (M=192, n=4): 0.519) by 7.1 points.
- MPNet at 32×: BAT-STE scores 0.582, beating the best post-hoc method (PQ (M=96, n=8): 0.562) by 2.0 points.

These gaps are large enough that no post-hoc configuration at any compression ratio dominates the training-time methods on these backbones. BAT-STE and BAT-atanh are therefore unambiguously Pareto-optimal at 32× for RoBERTa and MPNet.

5.6 Cross-Backbone Generalisability

Figure 5.8 reports *quality retention* (compressed MRR@10 divided by the same backbone's float32 MRR@10) rather than raw MRR@10. All three backbones have inherent retrieval capability after fine-tuning (Section 5.1), with BGE

the strongest. Normalising by each backbone's own float32 ceiling removes that absolute-quality gap.

The highest cross-backbone variance sits with BAT-STE (std 0.043) and BAT-atanh (std 0.075). As established in Section 5.3.1, the relative gain from these methods follows RoBERTa > MPNet > BGE. The remaining configurations, covering MRL truncation, INT8, TQ, PQ, post-hoc binary, and their non-BAT stacked variants, are largely invariant to the backbone.

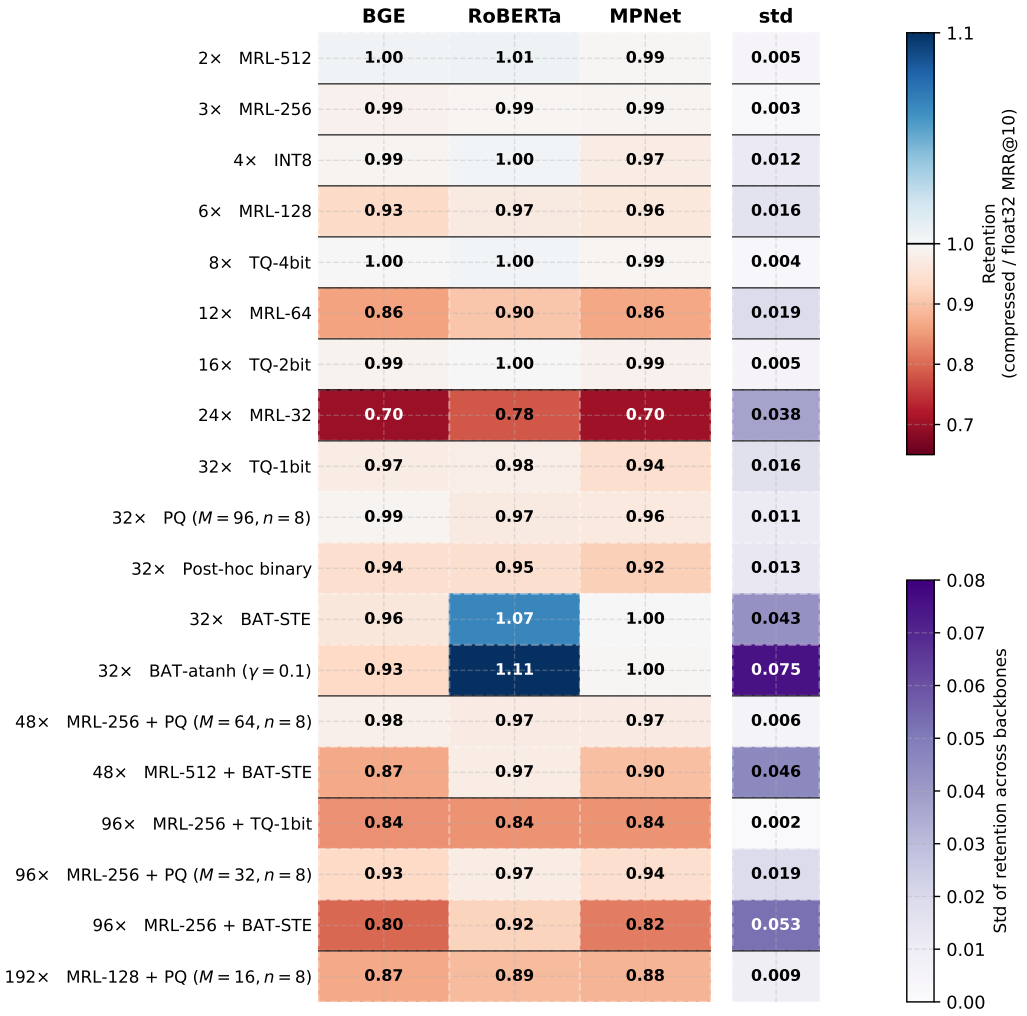

Figure 5.8: Quality retention (compressed MRR@10 divided by the same backbone's float32 MRR@10) for a representative set of configurations, ordered by compression ratio. The rightmost column shows the standard deviation of retention across the three backbones.

Chapter 6. Conclusion and Future Work

This thesis set out to compare embedding compression methods for dense retrieval on equal terms. Three encoder backbones with different retrieval priors were fine-tuned under identical conditions, evaluated on the same benchmark (NanoBEIR), and exposed to the same family of post-hoc, training-time, and stacked compression methods.

6.1 Conclusion

6.1.0.0.1 Post-hoc versus training-time compression. Training-time compression is not uniformly better than post-hoc compression; the winner depends on the backbone. BGE was pre-trained at large scale with a contrastive retrieval objective, so it begins from a retrieval-friendly representation. For BGE, post-hoc Product Quantisation (PQ) sits on the Pareto frontier at every compression ratio tested and beats training-time binarisation. RoBERTa and MPNet have no such retrieval pre-training; for them, training-time binarisation is on the frontier (especially at 32×) while no post-hoc method reaches it.

6.1.0.0.2 STE versus annealed tanh for binary embedding training. For training-time binarisation, the choice of gradient surrogate also matters and is itself backbone-dependent. In standalone training, STE is better on BGE

and MPNet, while annealed tanh is better on RoBERTa. In stacked training, annealed tanh combines with MRL much more cleanly than STE, beating MRL+BAT−STE on every backbone and matching or exceeding MRL+post-hoc binary on RoBERTa and MPNet.

6.1.0.0.3 Stacking complementary methods. Stacking dimensional reduction with precision reduction pushes the frontier well past what either method reaches alone. On BGE, MRL+PQ dominates standalone PQ in the 48× to 96× range, and MRL+TurboQuant stays usable through 96×. Not every combination helps: joint MRL+BAT training is worse than the sequential MRL→post-hoc binary path on BGE and MPNet. A likely cause is a gradient conflict: MRL needs the leading dimensions to carry informative float32 magnitudes, but the straight-through gradient discards those magnitudes. Annealed tanh avoids the conflict and restores the joint-training gain on the general-purpose backbones.

6.1.0.0.4 Cross-backbone generalisability. These findings transfer unevenly across backbones. Below 8×, the backbone barely matters: INT8 and 4-bit TurboQuant are near-lossless on all three, and when methods are ranked by quality the order is the same on every backbone. At 32×, the backbone takes over. The order among post-hoc methods stays the same (PQ > TurboQuant-1bit > post-hoc binary), but training-time methods switch position relative to post-hoc methods, and their effect flips sign: BAT pulls BGE below float32 while pushing RoBERTa above it. A study run only on a retrieval-specialised backbone

therefore does not generalise to general-purpose backbones at high compression (Section 5.6).

Taken together, the results in this thesis establish that no single compression strategy dominates across backbones or compression budgets. The right method depends on two things: whether the encoder has been pre-trained for retrieval, and the target memory budget. Controlled cross-backbone evaluation, of the kind conducted here, is a prerequisite for principled method selection in dense retrieval deployments.

6.2 Future Work

Several limitations constrain the scope of these conclusions and point to natural next steps.

6.2.0.0.1 Scale. NanoBEIR enabled rapid sweeps but its subsets hold only 2,000 to 6,000 passages each, and PQ codebook quality in particular is sensitive to corpus size. A follow-on evaluation on full BEIR or MTEB would test whether the conclusions transfer. The fine-tuning mixture is also fixed at a moderate size; a larger, more diverse mixture may change the gain from binarisation-aware training, especially on backbones without a retrieval prior. Every model is trained for a single epoch, and longer schedules, especially when paired with annealed tanh, may push the training-time frontier further.

6.2.0.0.2 MRL with recurrent binarisation. The stacked combinations evaluated here pair MRL with a single precision-reduction method (PQ, Tur-

boQuant, or binary). A natural extension is to stack MRL with the recurrent residual binarisation scheme deployed at Tencent [11], which encodes embeddings as a chain of binary stages and reaches a user-specified bit depth. Because the bit depth is chosen after training rather than baked in, the resulting pipeline gives post-hoc control over both the truncation dimension and the bit depth, a finer-grained quality−compression knob than any combination evaluated here.

6.2.0.0.3 Why BAT helps RoBERTa more than BGE. BAT raises RoBERTa above its float32 baseline at 32× but drops BGE below its own. One hypothesis is that BGE's float32 magnitudes already carry useful retrieval signal, so collapsing them to signs destroys information; RoBERTa's magnitudes are noisier, so the same collapse loses less. A direct test would add controlled noise to BGE's embeddings before binarisation and sweep the noise magnitude. If the hypothesis holds, BAT's relative quality on BGE should improve as the noise grows. A complementary analysis would compare the mean-pooled embeddings of BGE and RoBERTa before the sign() step, to identify the feature of BGE that binarisation removes.

References

[1] Faizan Ahemad. Quantization aware matryoshka adaptation: Leveraging matryoshka learning, quantization, and bitwise operations for reduced storage and improved retrieval speed. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*, 2025. URL `https://dl.acm.org/doi/10.1145/3746252.3761077`.

[2] Amazon Web Services. Amazon EC2 memory optimized instances (x2gd). `https://aws.amazon.com/ec2/instance-types/x2gd/`, 2024. Accessed: 2024.

[3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. MS MARCO: A human generated MAchine Reading COmprehension dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches (CoCo@NIPS)*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Zhangjie Cao, Mingsheng Long, Jianmin Wang, and Philip S. Yu. HashNet: Deep learning to hash by continuation. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 5608–5617, 2017.

[6] Ilias Caspar and Michael Dinzinger. CoRECT: A framework for evaluating embedding compression techniques at scale. *arXiv preprint arXiv:2510.19340*, 2025.

[7] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.

[8] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

[10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.

[11] Yukang Gan, Yixiang Ding, Qifan Guo, Jingang Wang, and Wei Wu. Binary embedding-based retrieval at tencent. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3966–3977, 2023.

[12] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6510–6526, 2021.

[13] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[14] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2010.

[15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[16] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611, 2017.

[17] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.

[18] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249, 2022.

[19] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[21] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[22] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, 2020.

[23] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–837, 2020.

[24] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023.

[25] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.

[26] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.

[27] Karsten Schrödter, Jan Stenkamp, Nina Herrmann, and Fabian Gieseke. Trainable bitwise soft quantization for input feature compression. In *Proceedings of the Third Conference on Parsimony and Learning (CPAL)*, 2026. URL `https://arxiv.org/abs/2603.05172`.

[28] Rushi Shah, Mingyuan Yan, Michael Curtis Mozer, and Dianbo Liu. Improving discrete optimisation via decoupled straight-through estimator. *arXiv preprint arXiv:2410.13331*, 2024.

[29] Aamir Shakir, Tom Aarsen, and Sean Lee. Embedding quantization. `https://huggingface.co/blog/embedding-quantization`, March 2024. Hugging Face Blog. Accessed: 2024.

[30] Samuel Sanford Shapiro and Martin Bradbury Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4):591–611, 1965.

[31] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, pages 16857–16867, 2020.

[32] Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908.

[33] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[34] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evalua-

tion of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

[35] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[36] Ellen M. Voorhees. The TREC-8 question answering track report. In *Proceedings of the 8th Text REtrieval Conference (TREC-8)*, pages 77–82, 1999.

[37] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the EMNLP 2018 Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, 2018.

[38] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[39] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[40] Ikuya Yamada, Akari Asai, and Hannaneh Hajishirzi. Efficient passage retrieval with hashing for open-domain question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 979–990, 2021.

[41] Jiwei Yang, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-Sheng Hua. Quantization networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7308–7316, 2019.

[42] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

[43] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In

The Thirteenth International Conference on Learning Representations, 2026. URL https://openreview.net/forum?id=tO3ASKZlok.

[44] Zeta Alpha. NanoBEIR: A collection of smaller versions of BEIR datasets. https://huggingface.co/collections/zeta-alpha-ai/nanobeir-66e1a0af21dfd93e620cd9f6, 2024. Hugging Face dataset collection.

[45] Biao Zhang, Lixin Chen, Tong Liu, and Bo Zheng. SMEC: Rethinking matryoshka representation learning for retrieval embedding compression. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. URL https://aclanthology.org/2025.emnlp-main.1332/.

Appendix A: Annealing Rate Ablation

A.1 Effect of γ on Annealed Tanh Binarization-Aware Training

Table 1 reports the mean MRR@10 across all 13 NanoBEIR subsets for each backbone model trained under three values of the annealing rate $\gamma \in \{0.05, 0.1, 0.2\}$. No single value of γ consistently outperforms the others across all backbones. BGE-base favours $\gamma = 0.2$ (wins on 8 of 13 subsets), while RoBERTa-base favours $\gamma = 0.1$ (7 of 13) and MPNet-base favours $\gamma = 0.05$ (6 of 13). The maximum MRR@10 spread within any backbone is 0.014 (RoBERTa), indicating that the results are not practically sensitive to this hyperparameter. We fix $\gamma = 0.1$ throughout as a principled default following [40].

Table 1: Mean MRR@10 across 13 NanoBEIR subsets for each backbone trained under three values of the annealing rate γ. Bold indicates the best value per row.

Backbone	$\gamma = 0.05$	$\gamma = 0.1$	$\gamma = 0.2$
BGE-base-en-v1.5	0.640	0.638	**0.648**
RoBERTa-base	0.576	**0.590**	0.580
MPNet-base	**0.585**	0.580	0.573

Appendix B: Statistical Significance Tests

B.2 Test Setup

The paired Wilcoxon signed-rank test, paired t-test, Shapiro-Wilk normality test, and Cohen's d are defined in Section 2.4.2. This appendix records only the choices specific to the evaluation pipeline.

All tests are two-sided with $\alpha = 0.05$. The Wilcoxon p-value is reported as the primary statistic, since per-query retrieval metrics are heavily non-normal. Win, tie, and loss counts are computed by direct per-query comparison of the two score vectors.

In addition to the per-subset tests, a pooled *mean* test is computed by concatenating per-query scores from all 13 NanoBEIR subsets into a single paired sample, preserving pairing within each subset. This pooled test assesses whether one model is significantly better than the other across the full evaluation set, rather than on any individual subset.

B.3 BGE: Pre-trained vs Fine-tuned at MRR@10

Per-query MRR@10 is recorded for both pre-trained BGE-base-en-v1.5 and its MNRL fine-tuned counterpart on the 13 NanoBEIR subsets. Pooling across subsets while preserving within-subset pairing yields the test sample summarised in Table 2.

The 75 % tie rate (490 of 649 queries) shows that for most queries the two models return identical top-10 orderings; the remaining differences are small and balanced in direction, leaving no statistically detectable shift in retrieval quality.

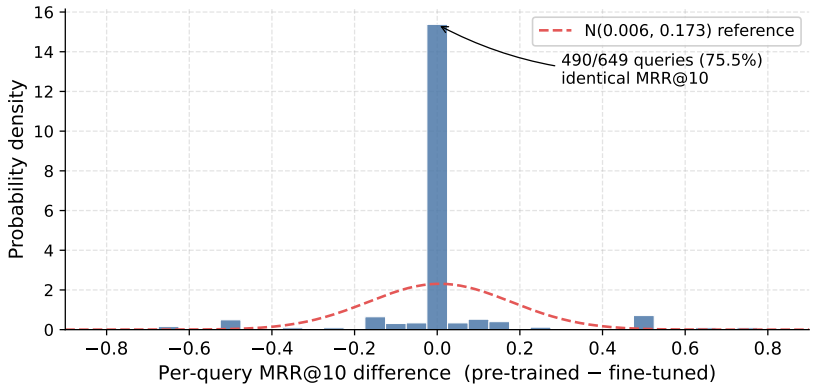

Figure B.1: Distribution of per-query MRR@10 differences (pre-trained − fine-tuned) for BGE-base-en-v1.5, pooled across the 13 NanoBEIR subsets ($n = 649$). The red dashed curve is the maximum-likelihood normal fit. The distribution is sharply concentrated at zero and the non-zero mass is roughly symmetric, making the normal fit a poor model and motivating the non-parametric Wilcoxon test reported in Table 2.

B.4 Binarisation-Aware Training Exceeds Float32 on RoBERTa

Per-query MRR@10 is recorded for the RoBERTa float32 fine-tune and for each of its two binarisation-aware variants (BAT-STE and BAT-atanh) on the 13 NanoBEIR subsets. Pooling across subsets while preserving within-subset pairing yields the test sample summarised in Table 3. Positive Cohen's d favours the binarised variant.

Both tests reject the null of zero median difference at $\alpha = 0.05$, and both effect sizes point the same way: the binarised variant scores higher than the

float32 fine-tune on average per query. The gain is therefore real, not a sampling artefact, and BAT-atanh shows the larger effect of the two.

Table 2: Paired significance summary for pre-trained vs MNRL fine-tuned BGE-base-en-v1.5 on per-query MRR@10, pooled across the 13 NanoBEIR subsets. Positive Cohen's d favours the pre-trained model.

Parameter	**Value**	**Description**
n	649	Paired per-query observations pooled across the 13 NanoBEIR subsets.
Wins	82	Queries where pre-trained MRR@10 > fine-tuned.
Ties	490	Queries where both models return identical MRR@10.
Losses	77	Queries where fine-tuned MRR@10 > pre-trained.
Cohen's d	$+0.035$	Effect size on the difference vector; well below the $\lvert d\rvert \approx 0.2$ small-effect threshold.
Shapiro–Wilk p	$< 10^{-3}$	Rejects normality of the paired differences; motivates the Wilcoxon test in preference to the paired t-test.
Wilcoxon p	0.486	Two-sided paired signed-rank test; fails to reject H_0 of zero median difference at $\alpha = 0.05$.

Table 3: Paired significance summary for BAT-STE and BAT-atanh against the RoBERTa float32 fine-tune on per-query MRR@10, pooled across the 13 NanoBEIR subsets.

Parameter	**BAT-STE**	**BAT-atanh**	**Description**
n	649	649	Paired per-query observations pooled across the 13 NanoBEIR subsets.
Wins	155	167	Queries where the binarised variant scores higher.
Ties	361	370	Queries where both models return identical MRR@10.
Losses	133	112	Queries where the float32 fine-tune scores higher.
Cohen's d	+0.117	+0.207	Effect size on the difference vector; both favour the binarised variant.
Shapiro–Wilk p	$< 10^{-3}$	$< 10^{-3}$	Rejects normality of the paired differences; motivates the Wilcoxon test in preference to the paired t-test.
Wilcoxon p	0.005	$< 10^{-3}$	Two-sided paired signed-rank test; rejects H_0 of zero median difference at $\alpha = 0.05$ in both cases.